

ImmerseDataXR: Advancing Data Comprehension with Immersive Visualizations

Master Thesis



Author: Georgios Marios Kokiadis (Student ID: 1224)

Supervisor: Prof. George Papagiannakis

Department of Computer Science
School of Sciences & Engineering
University of Crete
Heraklion, Greece

June 26, 2026

UNIVERSITY OF CRETE
COMPUTER SCIENCE DEPARTMENT

**ImmerseDataXR: Advancing Data Comprehension with Immersive
Visualizations**

Thesis submitted by
Georgios Marios Kokiadis
in partial fulfillment of the requirements for the
Masters' of Science degree in Computer Science

THESIS APPROVAL

Author: _____
Georgios Marios Kokiadis

Committee approvals: _____
George Papagiannakis
Professor, Thesis Supervisor

Panagiotis Tsakalides
Professor, Committee Member

Constantine Stephanidis
Professor, Committee Member

Departmental approval: _____
Polyvios Pratikakis
Associate Professor, Director of Graduate Studies

Heraklion, February 20, 2024

Abstract

The advent of immersive technologies like Virtual Reality (VR) and Augmented Reality (AR) has opened new frontiers in data visualization, particularly in handling complex datasets in fields such as biology. This thesis introduces a novel approach to data analysis through the development of an immersive visual analytics toolkit, designed to operate within VR and AR environments. The toolkit leverages these technologies to transcend the limitations of traditional 2D/3D visualization methods, offering a more immersive, naturally interactive and intuitive platform for data exploration. Central to this study is the application of the toolkit in a biological context, demonstrating its capability to enhance the understanding and interpretation of complex time series data. The research showcases how immersive visualization can lead to more insightful and efficient analysis, particularly in identifying patterns and anomalies that are not readily apparent in conventional visualization formats.

Declaration of Authorship

I, Georgios Marios Kokiadis, declare that this thesis titled, "ImmerseDataXR: Advancing Data Comprehension with Immersive Visualizations", and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Signature: Georgios Marios Kokiadis

A handwritten signature in black ink, appearing to read 'G. Kokiadis', with a long horizontal stroke underneath.

Date: June 26, 2026

Acknowledgements

I extend my deepest gratitude to my main supervisor, Prof. George Papiannakis, for his invaluable guidance, unwavering support, and the myriad of opportunities he has provided me with throughout this journey. His insights and expertise have been pivotal in making this work possible. I am equally grateful to my overseer, Dr. Manos Kamarianakis, whose assistance has been instrumental both mentally and technically. Beyond his technical expertise and innovative problem-solving, he has provided unwavering mental and emotional support that has been crucial during challenging times. His influence extends beyond the confines of this project, leaving an indelible mark on my professional and personal outlook. For his mentorship, patience, and profound impact on my journey, I am eternally grateful.

Additionally, I extend my heartfelt thanks to Dr. Daphne Bazopoulou for her exceptional support and guidance on the biological aspects of my research. Her expertise was crucial in enhancing my comprehension and application of biological principles. Additionally, my sincere appreciation goes to Nikos Falalakis, whose contribution of valuable data and insightful feedback was instrumental to the progression and enrichment of my work. Nikos's willingness to share his knowledge and resources played a significant role in the development of my thesis, offering perspectives that greatly contributed to the depth and rigor of my research. The generosity and dedication of both Dr. Bazopoulou and Nikos Falalakis have left a lasting impact on my academic journey, for which I am endlessly thankful.

Special thanks also to Prof. Yiannis Kamarianakis for his invaluable assistance with the Outlier Rank Algorithm, enhancing our research for the use case. His expertise significantly enriched the analysis and interpretation of data, contributing to the project's success.

Lastly, heartfelt thanks to my friends and family. Their infinite patience, love, and unwavering support have been the backbone of my journey. They may not all be computer scientists, but given the amount of tech talk they've patiently listened to, they're practically honorary experts by now!

Contents

List of Figures

1. Introduction

The advent of the digital era has precipitated an exponential increase in data complexity and volume, particularly in fields like biology, where the intricate nature of data necessitates advanced analytical approaches. This thesis presents a pioneering approach to data visualization in these fields, utilizing the untapped potential of Virtual Reality (VR) and Augmented Reality (AR) (Together often referred to as Extended Reality, XR).

1.1. Background & Motivation

While current XR visualization frameworks exist, they often lack the interactivity and immersion necessary for a comprehensive data analysis experience. This research contributes to this evolving field by developing a toolkit that enables intuitive and interactive data visualizations, tailored for XR environments.

One of the most compelling aspects of XR in data visualization is the ability to interact with data in a more natural and intuitive way. The tactile and immersive experience offered by XR allows users to engage with data spatially and kinesthetically, going beyond the limitations of traditional screen-based analysis. This hands-on interaction can lead to deeper understanding and insights, as it aligns with our innate spatial reasoning skills. In AR, the overlay of data visualizations onto the real world further enhances this experience, providing a more natural, less intrusive way of interacting with digital elements. By employing XR, the aim is not only to utilize three-dimensional representations but also to unlock a more embodied and intuitive form of data exploration, making complex data more accessible and understandable.

The motivation for this thesis arises from the limitations of traditional visualization tools when applied to multidimensional, complex, or plain large data. There is a significant need for visualization methods that accurately represent this complexity in a user-friendly manner, which is particularly crucial in fields where experts may not have extensive background in programming or 3D graphics. Investigating the potential of XR in data visualization is critical for the future of interactive data analysis. This research aims to empower a wide range of disciplines by facilitating the creation of intuitive and immersive data visualizations, thereby advancing the field of visual analytics.

1.2. Objectives & Scope

The primary objective of this research is to fundamentally enhance data comprehension across various domains through the development of an innovative toolkit, optimized for immersive visual analytics. By harnessing the transformative capabilities of XR, this study aims to revolutionize the traditional approach to complex data analysis. A significant focus of this endeavor is not just the creation of these tools, but also a comprehensive evaluation of their effectiveness.

To achieve this, specific objectives have been set: Firstly, to demonstrate the practical utility and adaptability of the developed toolkit by applying them to a diverse array of data visualizations. These visualization experiments serve as critical test cases to explore the full spectrum of the framework's capabilities, its limitations, and its potential in various contexts.

Secondly, a key aspect of this research is to assess the ease of use and flexibility of the toolkit. Given the diverse backgrounds of potential users, especially in fields like biology where programming and 3D graphics skills may be limited, it is essential that the toolkit be user-friendly and adaptable to various needs. The effectiveness of the toolkit and framework will be evaluated based on their ability to enable users to efficiently create and manipulate complex data visualizations with minimal technical hurdles. Focusing on the specific issue of enhancing the precision and efficiency of lifespan analysis in *C. elegans* survival assays, this addition aims to showcase the toolkit's capacity for real-time monitoring and early anomaly detection in complex biological datasets. By directly addressing a tangible scientific problem, this application not only tests the practical utility and adaptability of the toolkit but also highlights its potential to revolutionize traditional data analysis methods in biology.

By achieving these objectives, the research aims to make significant contributions to the field of visual analytics, providing powerful tools that enable professionals from various disciplines to engage with and interpret complex data sets more intuitively and effectively.

1.3. Methodology Overview

The methodology encompasses designing the XR visualization toolkit, developing algorithms for pre-processing data for visualization, and conducting a series of experiments to demonstrate the system's effectiveness and versatility. A suite of visualizations, each tailored to specific uses, was developed and

subsequently presented to experts in respective fields for assessment of utility and efficacy.

1.4. Thesis Structure

This thesis is organized into seven chapters, each focusing on a specific aspect of our endeavour into immersive visual analytics. Chapter 1 introduces the research, its motivation, and objectives. Chapter 2 reviews the current state of the art in immersive visualization technologies. Chapter 3 details the methodology behind the development and evaluation of the ImmerseDataXR toolkit. Chapter 4 showcases various visualization techniques developed using the toolkit. Chapter 5 presents the experimental results from using the toolkit with biological data visualization Use Cases. Chapter 6 discusses the implications of these findings in biological research as well as in the broader domain of Immersive Visual Analytics. Finally, Chapter 7 concludes the thesis, summarizing the key findings and suggesting future research directions.

2. State of the Art

2.1. Introduction to the State of the Art

The rapid advancement of technology in recent years has ushered in both opportunities and challenges in data analysis. Growing complexity and dimensionality in datasets necessitate innovative approaches to glean meaningful insights from them. Traditional 2D visualizations often fall short in representing the multidimensional structures of biological datasets. Consequently, the exploration of immersive technologies, such as Virtual Reality (VR) and Augmented Reality (AR), has emerged as a promising strategy to overcome the limitations of conventional data visualization, particularly in contexts that require extensive interaction with data.

2.2. Historical Context

Historically, scientific data visualization has been challenging due to its complex nature and the difficulty of intuitive human comprehension. Initial efforts in visualizing complex data sought to leverage immersive technologies, with significant milestones achieved using the CAVE System [?]. Notably, it facilitated visualizations of atomic motion and inter-molecular interactions [?]. The TIDE System [?], which surfaced later, allowed data retrieval and visualization from remote databases, supporting varied datasets and interactive collaboration with remote users (Figure ??). Furthermore, H. R. Nagel and colleagues proposed a customizable data representation approach [?] using the 3D Visual Data Mining System [?], offering users more flexibility in visualizing complex datasets to meet specific needs (Figure ??). These foundational efforts have addressed persistent challenges like remote and cross-platform collaboration, especially during times of limited internet connectivity, and have forged paths towards exploring user interaction with complex datasets in intuitive and effective ways.

2.3. Collaboration, Interaction & Customization

Immersive technologies, especially XR environments, have enhanced data visualization and analysis by providing interactive and immersive environments. While [?] highlighted VR's potential in improving accuracy and reducing mental demand in network visualization, [?] emphasized the importance of carefully

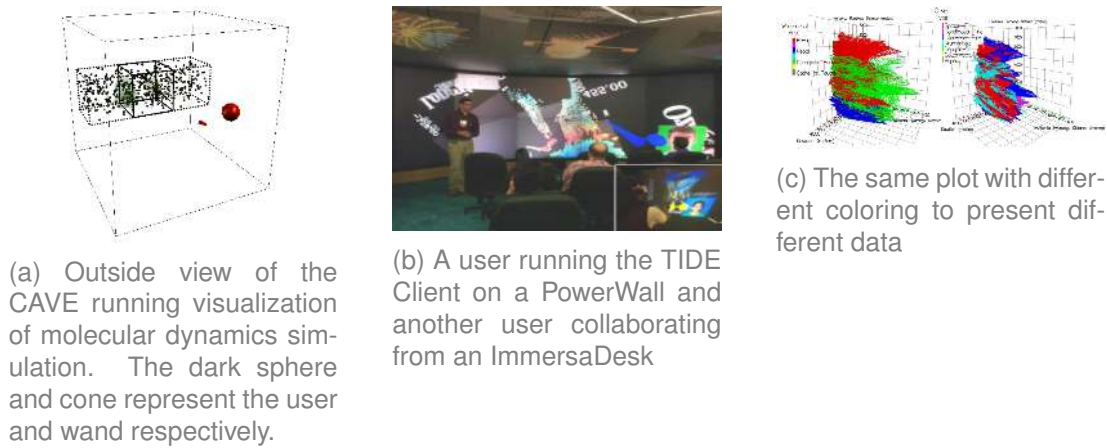


Figure 1: First attempts at Immersive Visualization

selecting display modalities to optimize data visualization within immersive analytics interfaces.

These virtual environments have also facilitated collaboration, even among geographically dispersed individuals. Huang [?] showcased VR's efficacy in collaborative learning, enabling real-time interaction and collaborative problem-solving among remote participants. Similarly, Lee et al. [?] demonstrated in their study the potential of immersive technologies in supporting collaborative visual data analysis. Their system, FIESTA, allowed co-located teams to use flexible visualization authoring tools, enabling them to structure their shared workspace in a virtual environment. This approach supported both individual and collaborative work, showing how 2D and 3D visualizations can be organized and interacted with in immersive spaces. Furthermore, Yang et al. [?] explored immersive collaborative sensemaking, comparing VR-based collaboration with conventional desktop conferencing. Their findings suggest that VR can simulate the physical experience of in-person collaboration more effectively, leading to more engagement and equal participation among remote collaborators. These virtual spaces not only facilitate visualization but also promote collaborative knowledge construction, aligning with global trends in remote interaction.

A pivotal strand of exploration within this domain pertains to the realm of AR and its capabilities in situating analytics within real-world contexts. The comprehensive work by Shin et al. [?] illuminated this domain, identifying and classifying 47 pertinent situated analytics systems. The systems were taxonomized across three vital dimensions: situating triggers, view situatedness, and data depiction, establishing a comprehensive framework to comprehend the nuances of immersive AR applications in analytics. Their ensemble cluster

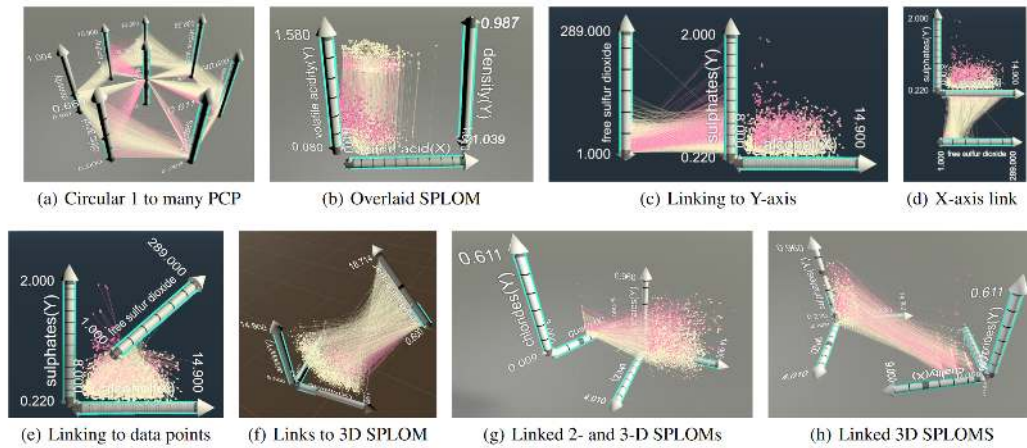


Figure 2: ImAxes. Each axis has data points that automatically link with other axes to generate complex plots.

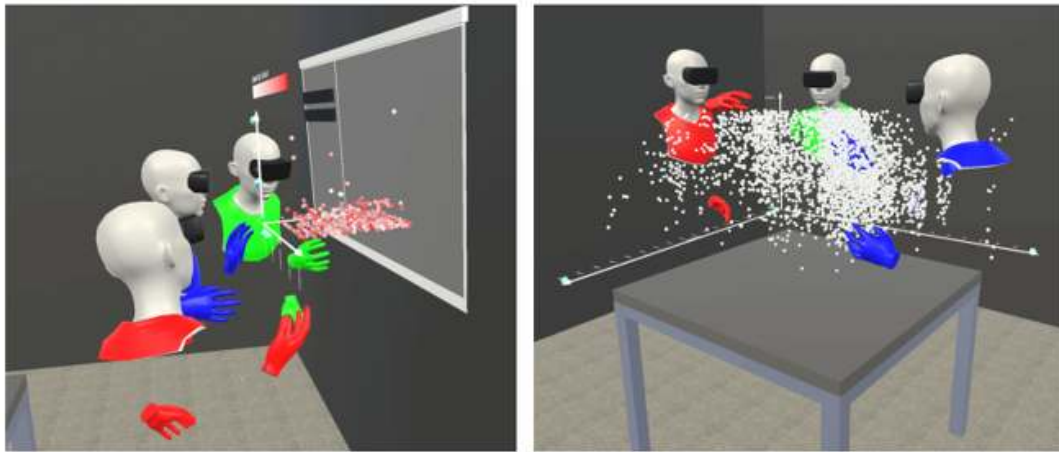


Figure 3: FIESTA, a collaborative visualization authoring tool.

analysis further delineated four archetypical patterns, yielding valuable insights and design guidelines that have potential ramifications for the design and application of AR in immersive analytics, particularly in tailoring the analytics to the user's physical locality and contextual environment.

Continuing in this vein, the development of user-friendly tools for immersive environments has also been crucial. As illustrated by [?], [?], and [?], the advent of interactive scientific data exploration tools and methods that facilitate easy crafting and exploration of visualizations in immersive spaces has become pivotal in enhancing user experience in data exploration and visualization. This amalgamation of technologies and methods underpins the ongoing evolution of immersive analytics, blending interactive, collaborative, and customizable experiences in data analysis and visualization.

2.4. Applications in Life Sciences

Biology and data visualization intersect to create both challenges and opportunities. Due to its complexity, biological data, especially from genomics and structural biology, demands innovative visualization and interpretation strategies. Recent advancements in biological data visualization have begun to challenge the traditional two-dimensional (2D) techniques. Gehlenborg [?] pioneered addressing the complex challenges posed by 'omics' data, focusing on generating clear and insightful visual representations while managing the inherent complexity of such datasets. Similarly, Weiskopf [?] (Figure ??) introduced a paradigm shift in uncertainty visualization by representing uncertainty in a three-dimensional (3D) space. Additionally, the integration of Virtual Reality in biology and data visualization, while still in its infancy, has achieved notable progress. Researchers have utilized VR to delve into biological structures, exploring intricate molecular designs [?] (Figure ??), complex protein interactions [?] (Figure ??), and detailed 3D models with unmatched performance and clarity [?]. Other studies, like [?], have explored oncology observation in cancer patients, providing enhanced insights into the relationships between genetic profiles, cancer types, and treatment options.

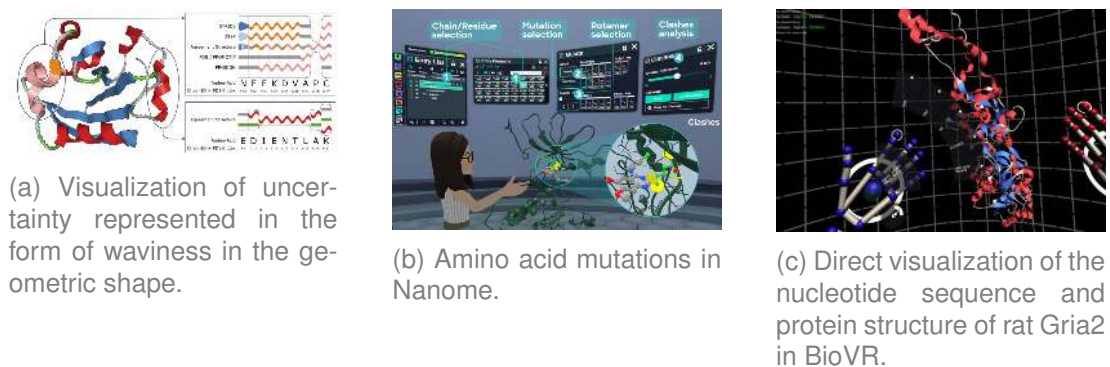


Figure 4: Visualisations of Biological Data

2.5. Conclusion of the State of the Art

Immersive visual analytics, particularly those utilizing XR technologies, offer profound investigative depth for biological data, enabling the identification of critical data points and revelation of patterns within complex, multidimensional datasets. The capabilities of immersive technologies, as established in the literature, provide a vital foundation for our research, especially in examining biological data and identifying anomalies therein. In summary, the current state of the art underscores the applicability and efficacy of immersive technologies

in analyzing complex data and provides a robust platform for their use in exploring the intricacies within biological datasets.

Despite these advancements, certain areas remain underdeveloped. There is a noticeable lack of generic immersive visualization toolkits that can be adapted across various research and development contexts. The field would benefit greatly from offerings that facilitate deeper interaction with data, which are not confined to a single use case or visualization type. Additionally, there is a need for more customization at the developer level to tailor these immersive technologies to specific research needs and workflows. Addressing these gaps would significantly enhance the versatility and effectiveness of immersive visual analytics in biological data analysis and beyond.

	Toolkit	Open Source	Collaboration	Various data types	Cross-Platform	Immersive Technologies
Huang, H. H. et al. (2023a)	No	No	No	No	No	Yes
Huang, W. et al. (2023b)	No	No	Yes	Yes	No	Yes
Lee, B. et al. (2021)	No	No	Yes	Yes	No	Yes
Yang, Y. et al. (2022)	No	No	Yes	Yes	Yes	Yes
Cordeil, M. et al. (2017)	Yes	Yes	Yes	Yes	No	Yes
Cordeil, M. et al. (2019)	Yes	Yes	Yes	Yes	No	Yes
Liu, R. et al. (2021)	No	No	No	Yes	No	Yes
Gehlenborg, N. et al. (2010)	Yes	No	No	No	No	No
Weiskopf, D. (2022)	No	No	No	No	No	No
Bennie, S. J. et al. (2023)	No	No	Yes	No	No	Yes
Zhang, J. F. et al. (2019)	Yes	No	Yes	No	No	Yes
Lau, C. W. et al. (2022)	No	No	Yes	No	No	Yes

Figure 5: Summary of features in State of the Art

3. Methodology

This research centers on the development and validation of an immersive visualization toolkit designed for enhanced data analysis across various domains. The primary innovation lies in the application of XR technologies to create an intuitive and interactive environment for data visualization. The goal is to transcend traditional data analysis methods by offering a more engaging and insightful visualization experience that leverages the immersive capabilities of XR.

The methodology employed in this study is twofold: firstly, it involves the careful design and implementation of the immersive visualization toolkit (ImmerseDataXR), ensuring that it is versatile enough to be applied to a wide range of data types and analysis scenarios. This includes the development of key features and functionalities that exploit the unique advantages of immersive technologies, such as spatial interaction, three-dimensional data representation, and real-time data manipulation capabilities.

Secondly, the research methodology includes the validation of this toolkit through a practical case study focused on biological data analysis. This case study serves as a proof of concept, demonstrating how the toolkit can be effectively utilized in a specific, complex domain. It provides an opportunity to assess the toolkit's practicality, user experience, and overall effectiveness in a real-world application scenario.

Through this approach, the research aims not only to contribute a novel tool to the field of data visualization but also to empirically evaluate its application, particularly highlighting its benefits and potential in enhancing data analysis processes. The insights gained from this study are expected to be valuable for future developments in immersive technologies and their application in various scientific and analytical contexts.

3.1. Development of ImmerseDataXR

3.1.1. Design and Implementation

The design and implementation of the immersive visualization tools were guided by the objective of creating a flexible and intuitive system for data visualization within XR environments. The foundational design principle was to emulate the functionality and ease of use found in matplotlib [?], a widely-used plotting library in Python. This approach structured the system to enable various types

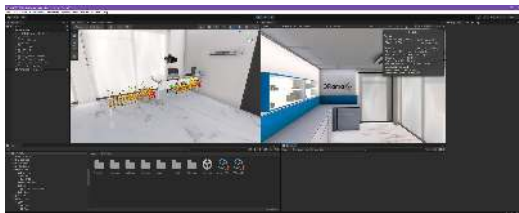
of data visualizations through a series of functions, each tailored to handle specific data types and visualization styles.

The technical backbone of the toolkit was built using the Unity Engine [?], a robust platform for creating interactive 3D content. Unity's capabilities in handling graphical elements and interactive features made it an ideal choice for developing a system that demands high visual fidelity and user interactivity.

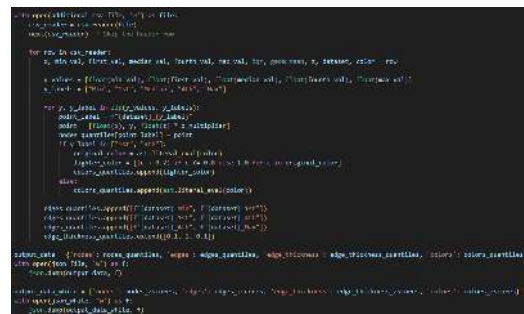
3.1.2. Integration of other Technologies

In demonstrating the versatility and robust capabilities of our immersive visualization toolkit, we strategically utilize the MAGES SDK [?]. This choice allows us to showcase the toolkit's adaptability to different SDKs and particularly highlight its efficacy in facilitating both singleplayer and multiplayer interactions. This integration serves as an exemplary platform to demonstrate these interactive features, effectively illustrating how our toolkit can be integrated into various XR and non-XR environments. Additionally, through the combined usage of the MAGES SDK and Photon Unity Networking [?], the ability to use the Toolkit in a co-operative, multiplayer scenario will also be explored.

To further test and explore ImmerseDataXR's functionalities, we employ a series of Python [?] scripts. These scripts are designed to generate diverse inputs, facilitating a comprehensive assessment of the toolkit's varied visualization capabilities. This methodological approach not only showcases the toolkit's seamless integration with external data processing tools but also provides a robust platform for evaluating its performance and flexibility. Such an evaluation is crucial for understanding the toolkit's ability to handle complex data sets in various domains, reflecting its potential for a broad spectrum of data analysis applications.



(a) Unity Game Engine.



(b) Python script in Visual Studio Code.

Figure 6: Technologies & Development Environments used for the Development of the ImmerseDataXR toolkit.

3.2. Toolkit Evaluation via Case Study

3.2.1. Case Study Selection Process

The selection of a case study in the field of biological data analysis is driven by multiple compelling factors. Biological data, by its nature, is inherently complex and multidimensional, often presenting significant challenges when attempting to represent it effectively in traditional two-dimensional formats. This complexity arises from the vast amounts of often multi-dimensional data points and the intricate relationships that exist within biological systems. These can often be challenging to encapsulate and interpret using standard visualization methods, be it three-dimensional or two-dimensional.

Moreover, the field of biology, compared to other disciplines like economics or computer science, often lacks advanced, specialized tools for data visualization. While biologists frequently rely on the R programming language [?] for data analysis and visualization, there remains a gap in terms of more advanced and immersive visualization tools. This presents a unique opportunity to explore and demonstrate the utility of our immersive visualization toolkit. By selecting a case study in this area, we aim to address these challenges head-on, showcasing how the toolkit can simplify and enhance the visualization and analysis of complex, high-dimensional data. The inclusion of R's data processing capabilities, combined with our toolkit's immersive visualization, could offer a significant advancement in how biological data is analyzed and understood.

In addition, biologists, as end-users, frequently do not possess extensive backgrounds in programming or data science, necessitating tools that are not only powerful in their analytical capabilities but also accessible and user-friendly. Therefore, this case study not only tests the technical proficiency of our toolkit in handling complex biological datasets but also its usability and adaptability in an environment where advanced computational skills are not a given.

Through this case study, we intend to demonstrate how our toolkit can bridge these gaps, offering biologists a means to visualize and analyze their data more effectively, thereby contributing novel solutions to the field of biological data analysis.

3.2.2. Application of Case Study in Immersive Analytics Toolkit

This subsection details how the ImmerseDataXR toolkit was applied in a specific case study involving a biological experiment on model organisms. The ex-

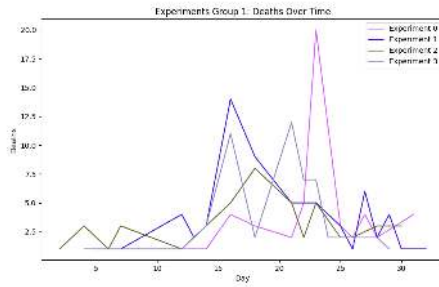
periment's goal was to observe the impact of various chemicals on the lifespan of model organisms, categorized into groups based on the type and concentration of chemicals used. The data collected from this experiment was in the form of a time series, recording daily deaths within each group.

C. elegans [?] [?] is a widely used model for survival assays due to its small size, rapid life cycle, and the ease of maintaining large populations. These assays, critical for understanding genetic and environmental factors affecting biological processes, often last 3-5 weeks and require daily assessments. However, factors such as contamination or environmental changes can compromise these experiments, leading to outliers in survival data. Traditionally, identifying such compromised datasets has been a time-consuming task, performed at the end of experiments.

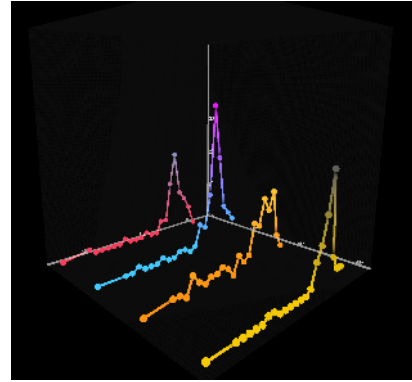
Lifespan Analysis The primary challenge lies in analyzing the effect of the chemicals on model organism lifespan was the visualization of multiple time series data concurrently. Traditional 2D methods often fall short in effectively displaying such complex data due to limitations in presenting multiple dimensions simultaneously. In contrast, our 3D approach within a XR environment enabled an intuitive and comprehensive visualization of these time series side by side. This immersive setup significantly enhanced the ease of navigation through the data, providing a clearer understanding of how each chemical treatment influenced the model organisms' lifespans (Figure ??).

Detecting Experimental Anomalies In biological experiments, particularly those involving time-to-event outcomes like survival assays, detecting anomalies early is crucial. Traditional methods rely on the expertise of biologists, who identify irregular patterns in data, such as unexpected spikes in death rates. However, this level of expertise is not always available, which can hinder the analysis and interpretation of survival data, especially in complex experimental setups like those involving the nematode *C. elegans* (Also referred to as model organism).

Improving Efficiency and Reproducibility in Biological Research ImmerseDataXR stands to significantly enhance the efficiency of biological experiments. It enables quicker responses to potential issues, reducing the time and resources spent on compromised experiments. By detecting compromised datasets earlier, during the assessment phase, it assists biologists in understanding discrepancies, ending faulty assays promptly, and designing new replicates in a



(a) 2D (R)



(b) 3D (ImmerseDataXR)

Figure 7: Model Organism Lifespan Plots using different visualization methods.

timely manner.

Enhancing User Accessibility A core objective of our visualization approach was to make complex data comprehensible even to those without extensive experience in biology. The intuitive interface and interactive capabilities of our XR toolkit are geared towards making advanced data analysis more accessible and engaging. This is particularly beneficial in educational or interdisciplinary contexts, where users might not possess specialized background knowledge in biological data analysis.

3.2.3. Collaborative Insights on Use Case

The contribution of biologists, particularly a professor with extensive experience in the field, played a pivotal role in shaping our research. Their insights were instrumental in bridging the gap between advanced data visualization technology and practical biological research needs. The professor provided crucial guidance on what both novice and seasoned biologists typically look for in data analysis, as well as current methodologies employed in biological research, which often rely on low-tech or traditional approaches. This knowledge was eye-opening, underscoring the potential impact and utility of our XR toolkit in revolutionizing data interpretation practices in biology.

Furthermore, the professor's input on interpreting complex biological data sets, such as identifying subtle yet significant patterns in model organism lifespan experiments, directly influenced the development and refinement of our visualization tools. Their expertise helped tailor our toolkit to meet the specific requirements and expectations of biological data analysis, ensuring that the tool was not only technologically advanced but also relevant and user-friendly.

for biologists.

Additionally, a student working under the professor's guidance was instrumental in providing actual experimental data from their ongoing research. This collaboration allowed us to test and validate our toolkit with real-world data, ensuring its effectiveness in practical scenarios. The student's role in gathering and supplying data was crucial in enabling us to fine-tune the toolkit's features to handle real biological datasets effectively.

3.2.4. Evaluation of the Visualizations

The observational approach in this study was predominantly exploratory, leveraging the immersive capabilities of the XR toolkit to facilitate an intuitive understanding of complex datasets. The method primarily involved hands-on demonstrations where participants interacted with the XR environment. This approach allowed users to navigate through the 3D space, offering a firsthand experience of the toolkit's functionality in visualizing time series data of model organism lifespans.

These sessions were informal and user-driven, focusing on the practical application of the XR tools rather than following a rigid observational protocol. Users were encouraged to freely explore the data, utilizing the XR toolkit to observe the effects of different chemicals on model organism lifespans and to identify any anomalies in the experimental data. This open-ended method of observation was chosen to closely mimic how end-users, especially biologists with varying levels of expertise, would interact with the tool in a real-world scenario. We then incorporate all given feedback into improving the visualizations using ImmerseDataXR, to complete the feedback loop for the development of the specific use case.

3.2.5. Validation and Application of Visualization Methods

The data analysis in this study was twofold. Initially, we worked with known datasets to validate the toolkit's ability to accurately visualize standard experimental data without environmental interference. This step was crucial to establish a baseline for the toolkit's effectiveness in standard conditions.

Subsequently, we analyzed datasets that were known to have been affected by environmental issues, thereby compromising the experimental outcomes. The algorithms developed for this use case, were applied to these datasets. The effectiveness of our method was inferred from the toolkit's ability to flag these

compromised datasets as outliers. This practical, results-oriented approach to data analysis allowed us to assess the toolkit's functionality in real-world conditions, particularly its capacity to identify anomalies that would typically require a high level of expert interpretation.

3.3. Ethical Considerations and Limitations

3.3.1. Ethical Aspects

While our research primarily focuses on the development of technological tools for data visualization, it is imperative to acknowledge and address the ethical considerations involved, particularly given the biological nature of the data used. First and foremost, the confidentiality and privacy of the data were maintained at all times.

We ensured that all data handling procedures adhered to the ethical guidelines and standards set forth by relevant institutional and scientific bodies. This includes obtaining necessary permissions and approvals for the use of the data in our research.

Furthermore, in our collaboration with biologists, especially in instances where we received direct input and data, we maintained transparency about the aims of our research and the intended use of the information provided. All parties involved in the study were fully informed about the nature of the research and the potential applications of the toolkit.

As this research did not involve human or animal subjects directly, the primary ethical considerations were centered around data integrity and the responsible use of information. We took great care to ensure that our research practices did not misrepresent or misuse the data in any way.

In summary, ethical considerations in our research were predominantly focused on data confidentiality, adherence to standard research protocols, transparency with collaborators, and responsible data usage. These measures were crucial in ensuring the integrity and ethical soundness of our research process.

3.3.2. Limitations

This research, while making significant strides in the field of immersive data visualization, encounters several inherent limitations that are important to acknowledge for a well-rounded understanding of its scope and application.

Data Multidimensionality Constraints One such limitation pertains to the multidimensionality of data from the model organism lifespan experiments. The inability to record additional dimensions of data was a constraint beyond our control. The lack of advanced sensors and the need to avoid compromising the integrity of the experiments limited the collection of more comprehensive data. In biological experiments, acquiring extra data often requires intrusive methods, such as sacrificing some of the model organisms, which could skew the experiment results. Furthermore, the scope for incorporating additional bio markers for enhanced multidimensional analysis was constrained, limiting the exploration of the full potential of immersive analytics in this context.

Comparative Analysis Challenges Another limitation was the challenge in comparing a broad range of experiments. Many of the experiments dealt with vastly different variables, rendering them scientifically incomparable. This limitation was compounded by the time and resources required to initiate and conclude new experiments. Given the lengthy duration of biological experiments, which can span several weeks, initiating new experiments to generate comparable data was not feasible within the time constraints of this research.

Focus on Visualization and Environment Optimization The primary focus of the research was on developing and optimizing the visualization toolkit for XR environments. As a result, less emphasis was placed on creating tools for data preprocessing. Developing such tools would have involved addressing a separate set of challenges and objectives, diverging from the main goal of enhancing immersive data visualization. Therefore, the preprocessing of data, an essential step in preparing datasets for immersive analytics, was not extensively covered in this research, leaving room for future exploration and development in this area.

4. Visualizations in ImmerseDataXR

In this section, we will go over on a comprehensive exploration of the visualizations possible within the ImmerseDataXR toolkit. ImmerseDataXR offers a suite of innovative and interactive visual tools specifically designed to leverage the immersive potential of XR environments. We will delve into the intricacies of each visualization, discussing their unique implementation details, the user interactions they support, and the scenarios where they are most effectively employed. The aim is to provide not only a technical overview but also insights into the design philosophy and practical applications of these visualization techniques in various domains.

4.1. Plot Viewport

ImmerseDataXR introduces the Plot Viewport, a dynamic and interactive component that revolutionizes the way data is visualized. This feature transcends the concept of a static display, offering users a flexible and intuitive canvas for exploring their data. Integral to the design of ImmerseDataXR, the Plot Viewport's capabilities are embedded in all the visualizations we will be discussing. It forms the backbone of our visualization approach, ensuring that each data representation benefits from the same level of interactivity, flexibility, and user-centered design.

4.1.1. Translating and Scaling the Data

The Plot Viewport in ImmerseDataXR does more than just show data; it lets users actively manipulate data in a 3D space. This feature enables users to move and resize data, helping them understand complex datasets from different angles and scales. This kind of interaction leads to a deeper and clearer understanding of complex, multidimensional data, making the Plot Viewport a practical tool for in-depth data analysis.

Users can easily move data around in the Plot Viewport using the 'grab' (Figure ??) button on their VR controllers and use their hand to drag the data around. For resizing the data, they can use the 'Trigger' button (Figure ??) – pulling the controller upwards makes the data bigger, and moving it downwards makes it smaller. These straightforward controller gestures make it easy for users to interact with their data in the VR environment.

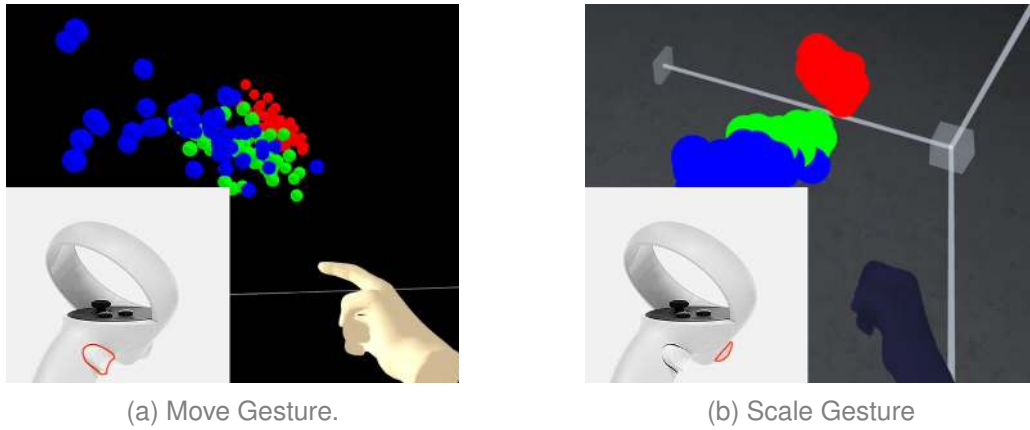


Figure 8: Gestures for transforming data in the Plot Viewport.

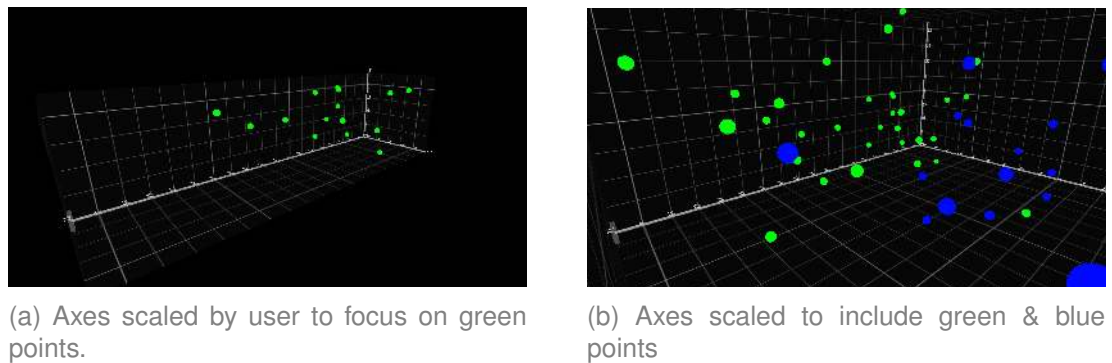
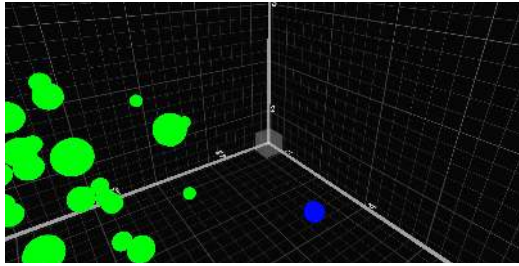


Figure 9: Customizable Plot Dimensions.

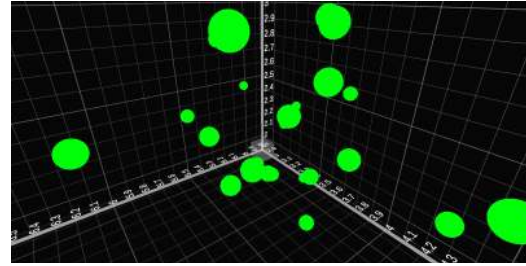
4.1.2. Editing the Viewport

Users can dynamically adjust the size of each axis via direct interaction, by grabbing and moving the end of the axes, thus enabling precise focus on specific data segments (Figure ??). The viewport includes axis numbers and grid lines, providing a familiar plotting view. These elements adapt in real-time to changes in data scaling and positioning (Figure ??), merging traditional plotting comforts with advanced 3D visualization. The users can also move the plot around their virtual or real environment by grabbing and moving the origin of the axes.

Furthermore, after customizing the axis dimensions to suit their specific data analysis needs, users can utilize a convenient auto-fit feature. With the simple press of a button, the Plot Viewport automatically adjusts to optimally fit the data within the newly defined dimensions (Figure ??). This auto-fit functionality ensures that the visualized data is always presented in the most effective manner, tailored to the current viewport settings.

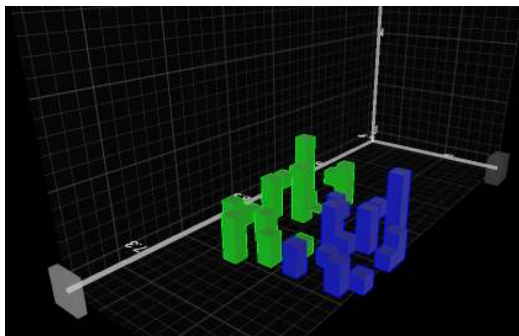


(a) Data Scaled normally.

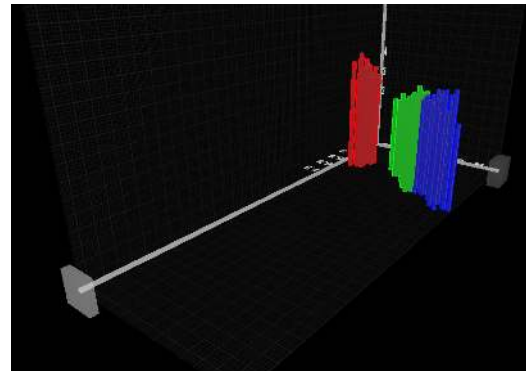


(b) Data Moved, and Scaled up.

Figure 10: Adaptive Grid Lines & Axis Numbers in Plot Viewport



(a) Data Points Before. Only some Blue & Green points are visible.



(b) Data Points After. All points are visible.

Figure 11: Automatically Fitting Data in Plot Viewport.

4.1.3. Investigating Specific Data Points

One of the interactive features of ImmerseDataXR is the capability for users to examine individual data points closely. By hovering over a particular data point within the visualization environment, users can access additional information about that point. This functionality is linked to a designated field in the input data, named "label." (Figure ??)

The "label" field is designed to be versatile, allowing users to input a wide range of information that they deem pertinent to each data point. This could include detailed descriptions, numerical values, categorical information, or any other relevant data annotations. When a user hovers over a data point, the information from the "label" field is displayed, providing immediate and context-specific insights. This feature enhances the user's ability to conduct a thorough and detailed analysis of the data set, making it easier to understand and interpret the significance of individual data points within the larger context of the data as a whole.



Figure 12: Hovering your hand on a data point.

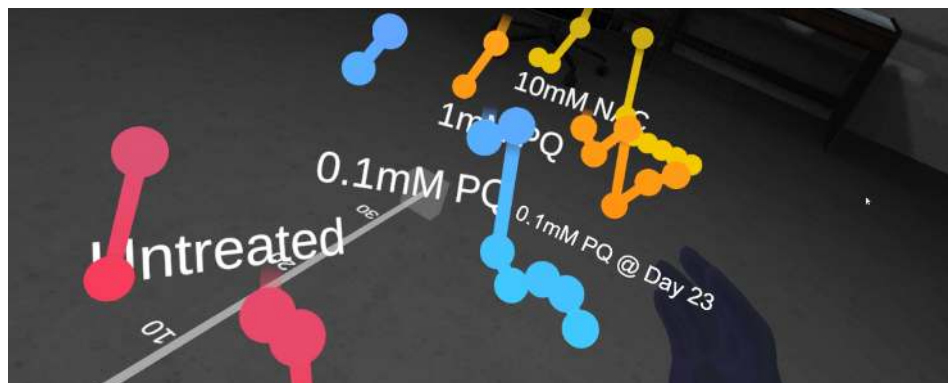


Figure 13: Standalone Labels alongside data points and hover Labels.

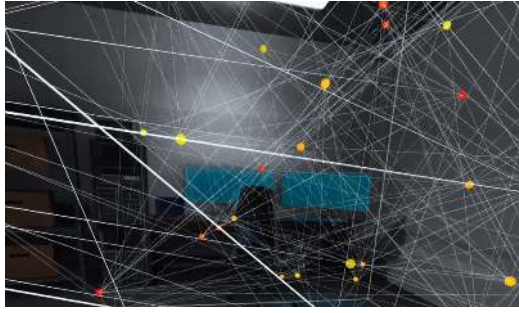
4.1.4. Standalone Labels

In addition to interactive data point investigation, ImmerseDataXR offers the functionality of standalone labels within the visualization environment. These labels serve as informative markers or notes within the plot, aiding in the user's comprehension and analysis of the visualized data. (Figure ??)

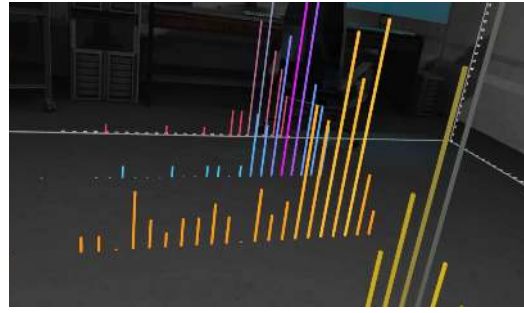
Standalone labels are created and customized directly in the input file. Users can add annotations, explanatory notes, or any other pertinent information as labels in the visualization. This feature is particularly useful for marking significant areas or points in the data, providing contextual explanations, or guiding the viewer through the dataset. The inclusion of these labels in the plot enhances the overall clarity and utility of the visualization, making it more informative and user-friendly, especially in complex datasets where additional guidance or information is beneficial.

4.1.5. Data Dive Mode

The Data Dive Mode, represents a transformative feature within ImmerseDataXR, offering users an immersive and in-depth data exploration experience.



(a) Complex Graph Plot.



(b) Timeline Visualization via Bar Plot.

Figure 14: Data Dive Mode in various Plots.

By activating this mode, users can step beyond the conventional confines of a fixed viewport, entering a state where the data envelops them in a complete 360-degree surround view.

Upon pressing a designated button, the Data Dive Mode is engaged, seamlessly transitioning the user's environment such that the data no longer resides within a bounded frame but instead extends all around them. This mode effectively turns the entire XR space the user occupies into an expansive, interactive data viewport.

In Data Dive Mode, users retain the ability to manipulate data, including moving and scaling operations, but these interactions take on a new dimension. The data can be arranged and examined from all angles, enabling a comprehensive and detailed analysis. This mode is especially advantageous for dissecting complex, large datasets, as it allows users to literally walk through their data, gaining insights from an up-close and personal perspective.

This immersive approach to data visualization not only enhances the user's ability to perceive intricate data patterns but also fosters a deeper connection with the data. By being surrounded by the data points, users can engage with their dataset in an intuitive and natural manner, akin to navigating a real-world environment but within the context of their data landscape.

4.1.6. Enabling Co-operative Exploration

The immersive visualization toolkit offers the capability for online cooperative exploration by integrating scripts from the MAGES SDK in conjunction with the Photon Unity Networking (PUN) package. This feature is particularly designed for facilitating collaborative analysis and data exploration in virtual environments. The Plot Viewport, central to the toolkit's design, has been specifically developed with cooperative interaction in mind, ensuring that the setup

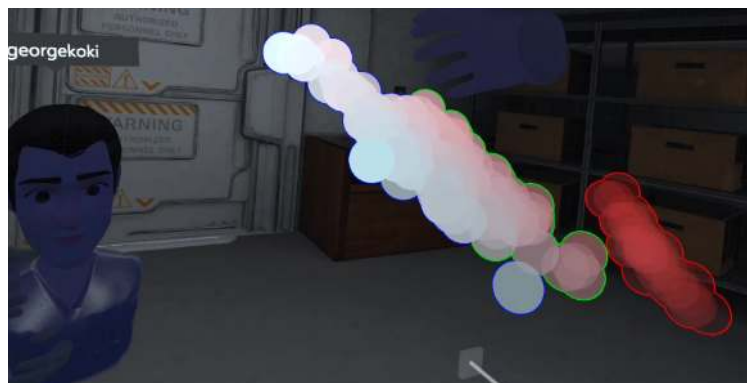


Figure 15: Co-operative Data Exploration.

process is straightforward and user-friendly. The incorporation of these cooperative functionalities significantly enhances the toolkit's utility, opening avenues for collaborative research and interactive learning experiences in immersive environments.

4.2. Plot Types

In ImmerseDataXR, users can engage with two primary plot types that cater to different visualization needs and preferences: the Prefab Plot and the Connected Plot. While these plot types share some foundational similarities, they each possess distinct features and require differently formatted input. To showcase the versatility of these plot types, we offer several examples that underscore their flexible and powerful nature. These instances reveal how effectively both plot types can be customized and adapted to suit a wide range of data visualization needs within the ImmerseDataXR framework.

Alongside the primary plot types, ImmerseDataXR also encourages innovation and customization through its extensible design. Both the Prefab Plot and the Connected Plot are built upon a flexible interface provided by the toolkit. This interface allows developers to craft additional visualization types beyond the core offerings. This aspect of ImmerseDataXR is pivotal in its utility, as it provides a foundation for developers to experiment with and introduce new visualization methods. By enabling such adaptability, the toolkit not only caters to a broad spectrum of current visualization needs but also paves the way for future advancements and novel applications in the realm of immersive data analytics.

For Instructions on how to set these Plots up in a Unity environment, see Appendix ?? on page ??.

The Class Reference needed for creating other plot types, is in Appendix ?? on page ??.

4.2.1. Prefab Plot

The Prefab Plot serves as a foundational element in ImmerseDataXR's visualization toolkit, offering a versatile and user-friendly platform for diverse data representation needs. Characterized by its rapid customization and adaptability, the Prefab Plot allows users to present a set of data points with a variety of prefabricated shapes, such as spheres, cubes, or custom 3D models. Additionally, users have the flexibility to customize the color of each individual data point, providing an added layer of insight and allowing for a more detailed and nuanced analysis of the data set.

At its core, the Prefab Plot is designed to handle a wide range of data types, from simple point representations to more complex data structures. Its intuitive

interface and customizable features make it an ideal choice for both novice and experienced users. Whether it's for scientific research, economic analysis, or educational purposes, the Prefab Plot provides a solid base for effective data visualization, ensuring clarity and insight in the interpretation of complex datasets.

Scatter Plot This Scatter Plot example in ImmerseDataXR functions similarly to traditional 3D scatter plots, offering a straightforward and effective way to visualize three-dimensional data. Users can input a series of 3D points, with each point customizable in color and label, making these plots ideal for datasets where differentiation and detail are key.

Commonly used in scientific and technical fields, Scatter Plots are effective for visualizing and analyzing spatial relationships and patterns. They are particularly useful in disciplines like physics, engineering, and environmental science, where understanding the distribution and interaction of variables in three-dimensional space is crucial.

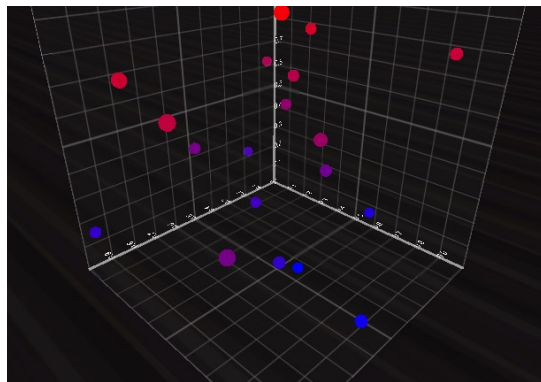


Figure 16: Scatter Plot with Random Points, where the color is more red the higher the value of the Y axis is in the ImmerseDataXR Plot Viewport.

Bar Plots Bar Plots in ImmerseDataXR are akin to the well-known bar plots in Python's matplotlib library, offering a familiar and effective method for visualizing data. These plots are ideal for representing categorical data with rectangular bars, where the length of each bar is proportional to the value it represents.

Just like in matplotlib, Bar Plots in ImmerseDataXR are versatile and can be used for a variety of applications, from simple data representation in business analytics to more complex data in scientific research. They are particularly useful for comparing different groups or tracking changes over time, making

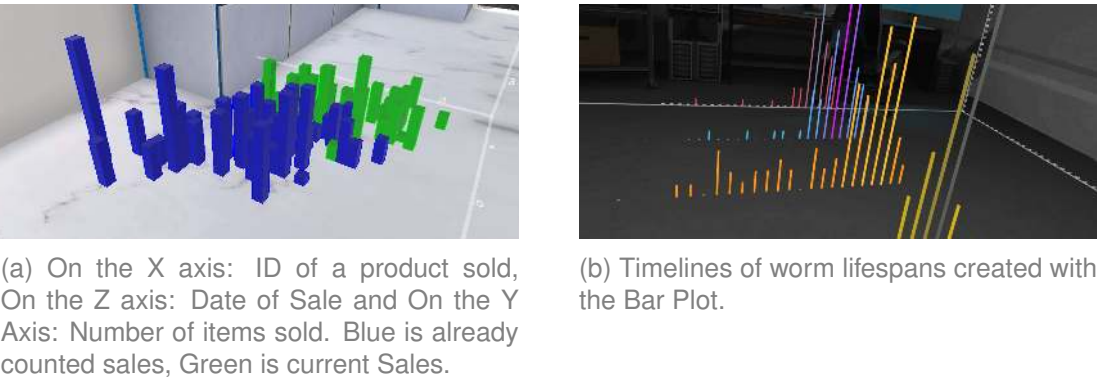


Figure 17: Bar Plots in ImmerseDataXR Plot Viewport.

them a valuable tool in fields such as economics, sociology, and environmental studies.

Column Customization Plot The Column Customization Plot in ImmerseDataXR, is an advanced extension of the Prefab Plot, and introduces a user-centric interface designed to handle datasets with high dimensionality. This plot type enables users to interactively select which data set columns correspond to various visualization attributes, facilitating the exploration of up to six dimensions simultaneously.

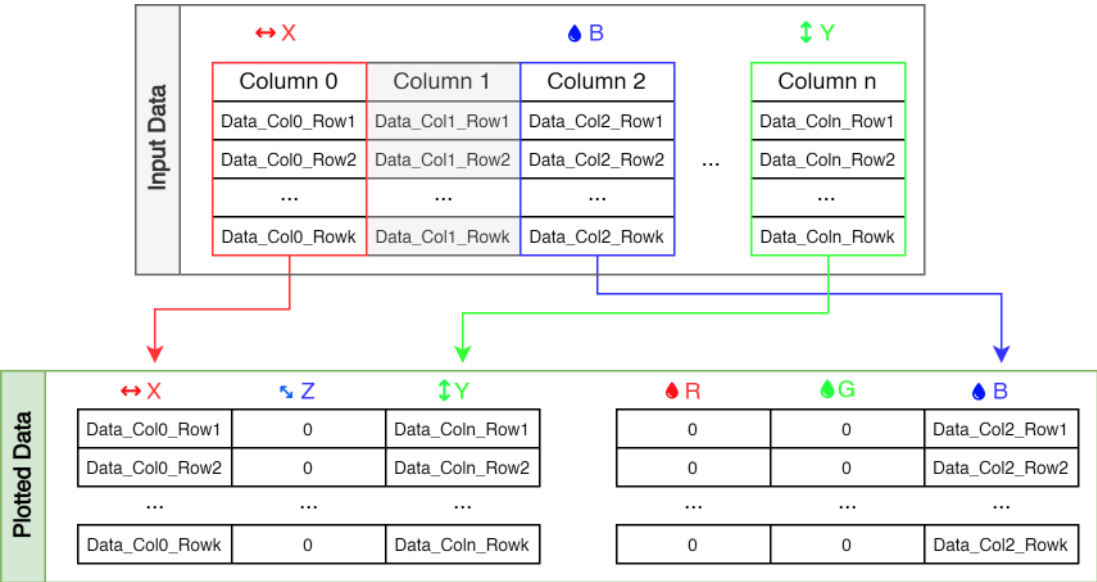


Figure 18: Flow of data in the Column Customization Plot.

Users can, for instance, assign column zero to the x-axis, column two to control the red value of the points, and column 'n' for the y-axis. Unspecified columns default to zero, simplifying the visualization process (Figure ??). This level of user-directed customization is particularly valuable for datasets with multiple dimensions, as it allows users to uncover hidden insights that might be

obscured in less flexible visualization formats.

The presentation and testing of this plot is achieved using the iris data set [?].

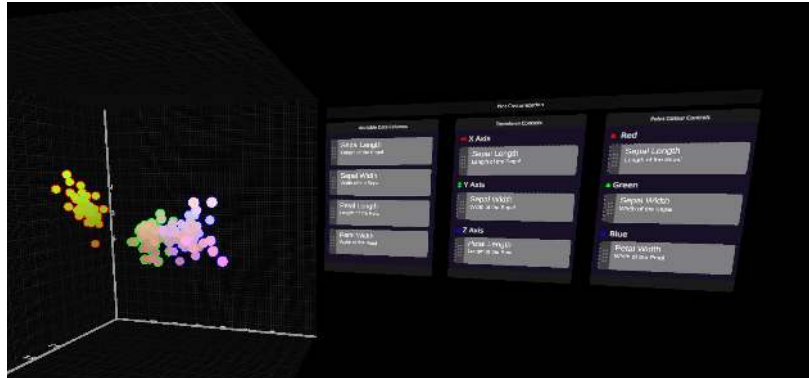


Figure 19: Column Customization Plot in ImmerseDataXR Plot Viewport. Using the iris data set.

The Column Customization Plot’s capability to provide direct access and manipulation of multiple dimensions simultaneously is a game-changer in data analysis. It offers a unique opportunity to delve into complex datasets, revealing patterns and correlations that could remain hidden without this degree of interactive and multidimensional exploration. By granting users the power to easily navigate and tailor their visualization approach, ImmerseDataXR paves the way for more profound and insightful data interpretations.

This Example incorporates a user-friendly interface with three distinct windows: one shows available data columns, the second is dedicated to the X, Y, and Z axes, and the third is for adjusting the data points’ colors using Red, Green, and Blue values. This setup allows users to intuitively customize their data representation by simply dragging and dropping columns into the respective axes and color windows. This interactive method of customization streamlines the process, making it an efficient and user-friendly approach for exploring complex datasets.

4.2.2. Connected Plot

The Connected Plot in ImmerseDataXR, while similar to the Prefab Plot in accepting color customizations and using 3D prefabs, introduces an additional feature of connecting data points with lines, or edges. A key aspect of this plot type is the ability to adjust the thickness of these lines directly from the input file, allowing for a visual representation that corresponds to the relationship’s intensity or other data-specific metrics.

Further enhancing its customization capabilities, the Connected Plot also al-

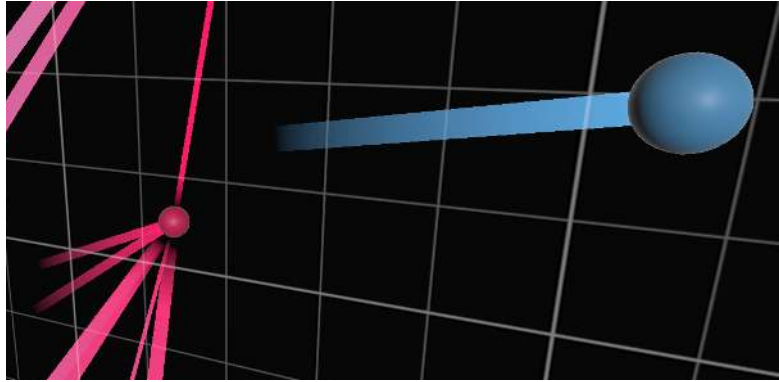


Figure 20: Connected graph nodes with connections to off-screen nodes.

allows for the coloring of the connecting lines based on the properties of the nodes they link. This means that lines can have a static white color, adopt the color of the connected nodes and fade the colors between the two nodes, or get colored based on their thickness defined by the input file. This feature is particularly useful for customizing the connections between points, making it easier to discern patterns and relationships in complex datasets, such as in network analysis or relational data visualization. Additionally, each line can present to the user the numerical value of the weight it has via text, for more accurate and deeper insights.

One thoughtful design detail addresses the visualization of connections when one of the data points lies outside the current plot viewport. When this occurs, the line connecting the in-view data point to the out-of-view point doesn't simply disappear; instead, it gracefully fades while maintaining its directional orientation.(Figure ??) This fading effect, coupled with the line's maintained trajectory, provides a clear visual cue of where the out-of-view point is located relative to the visible data. This subtle yet effective design choice aids users in understanding the topological layout of the entire dataset, ensuring that the spatial relationships and connections remain comprehensible even when some data points are not directly visible in the viewport. By preserving the context of these connections, the Connected Plot enhances the user's ability to grasp the complete structure of the data, regardless of the current focus area within the visualization environment.

Graph Visualization In the Connected Plot's application for graph visualization, users can display a connected graph with the option to color nodes based on their connectivity. For instance, a red-to-blue gradient might be used to reflect the number of connections each node has.(Figure ??) However, it's important to note that this coloring scheme is not mandatory. Users have the

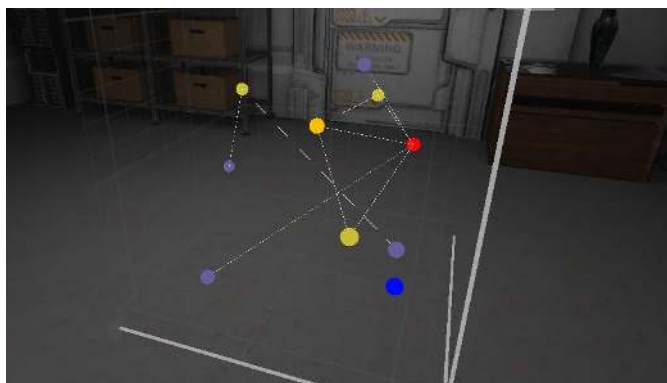


Figure 21: Connected graph with nodes colored based on the number of connections.

freedom to assign colors to nodes in any manner they prefer, depending on the specific details or attributes they wish to code into each node.

Graph visualizations in 3D are useful in various fields, including social network analysis, biological networks, and complex system modeling in areas like physics and computer science. They offer a way to visually represent the structure and connections within a graph, aiding in the analysis and understanding of complex network relationships.

Protein Relationship Visualization For the Bio+MedVis Challenge [?], which involved complex proteomic data analysis and drug response across diverse cell lines, a version of the Connected Plot was developed as an interactive visualization tool. This tool was designed to showcase the relationship between the proteomic profiles and drug response data [?].

Using the Connected Plot, each protein in the cell line was represented as a node. The drug responses were visualized as connections between these nodes, where the thickness indicated the strength of the drug response. This setup allowed users to visually explore the relationships between the proteomic makeup of the cell lines and their responses to different anti-cancer compounds. When hovered, the nodes present the user with the name and details of the protein that is represented via that node.

Due to the especially large size of the data set, a handful of relationships are picked to be visualized at random. This can of course be adjusted via the input file, which can include any proteins the user wishes to see.

The interactive nature of the Connected Plot enabled users to engage with the data directly, facilitating a more intuitive understanding of complex relationships. It provided a means to observe patterns and correlations within the dataset, such as identifying cell lines with similar proteomic profiles and drug

responses, or uncovering unique responses within certain cell types.

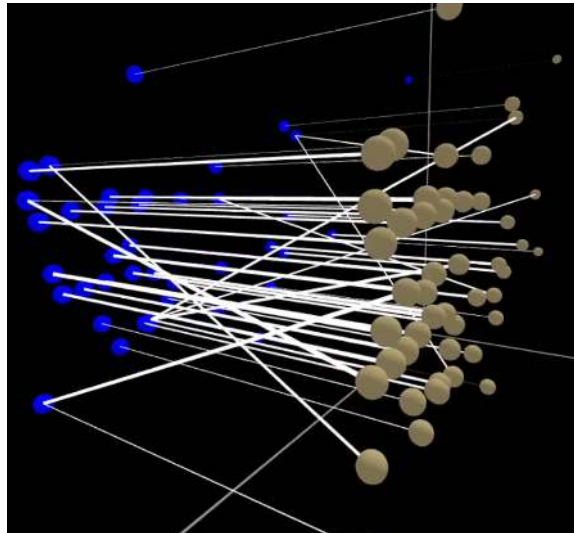


Figure 22: Protein Relationships in the Bio+Vis Challenge Data set.

Time Series Visualization A possibly unorthodox use case for the Connected Plot is time series visualization (Figure ??). Time Series are practical for various applications, such as monitoring stock market trends, tracking environmental changes, or observing patient health metrics over time. In this visualization, each time point is represented as a node, with edges connecting them in sequence. This format allows for a clear depiction of how data evolves, making it easier to spot trends and anomalies.

Representing a set of time series in 3D, addresses common challenges found in traditional 2D visualizations. In 3D, multiple time series can be spatially separated, avoiding the overlap and confusion that often occur in 2D plots. This separation becomes increasingly valuable as the number of time series grows, where 2D representations can become cluttered and impractical.

We use this type of visualization extensively in our Experimental Use case regarding worm lifespan.

Box Plot Visualization Using Connected Graphs One innovative application of the connected graph within our visualization toolkit is the creation of box plots. This approach utilizes the versatile properties of the connected graph to depict the quintessential elements of a box plot.

By manipulating the line thickness in the connected graph, we can effectively construct the primary components of a box plot. The central 'box', representing the interquartile range (IQR) – which includes the median, upper quartile, and

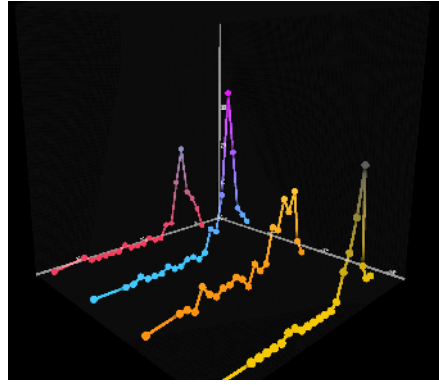
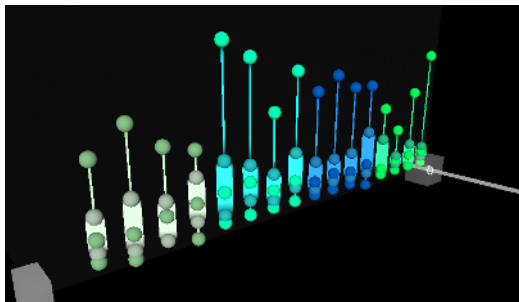
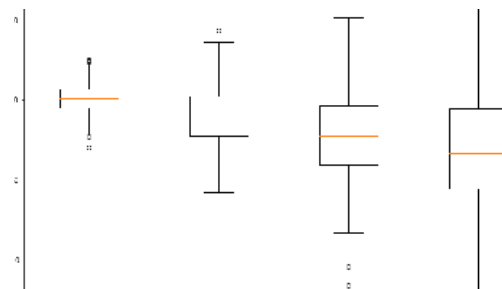


Figure 23: Time Series visualization using a connected graph plot.



(a) Box and Whisker Plot Visualization using a connected Graph Plot.



(b) Box and Whisker Plot in 2D.

Figure 24: Box and Whisker plots comparison.

lower quartile – is depicted using lines of higher thickness. This visual representation effectively encapsulates the core distribution of the data within each time series.

To represent the 'whiskers' of the box plot, which extend to the minimum and maximum values, thinner lines are employed. These whiskers provide a visual indication of the range of the data, highlighting the extent of variability outside the central IQR.

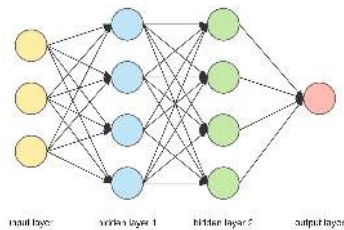
This creative use of connected graphs to generate box plots offers a unique way to visualize key statistical measures in a 3D environment. It allows users to grasp the distribution characteristics of each time series at a glance, enhancing the interpretability of complex datasets. This method not only preserves the traditional understanding of box plots but also brings a novel dimension to it by leveraging the interactive and immersive capabilities of 3D visualization.

Machine Learning Hidden Layers Visualization The Connected Plot is effectively utilized for visualizing hidden layers in machine learning algorithms, along with the weights connecting these layers. This visualization method offers a unique perspective for investigating how weights change in response to

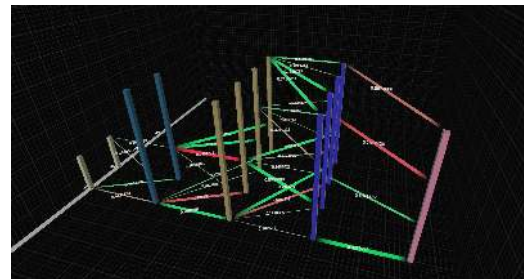
different inputs (Figure ??). It serves as a valuable tool both for educational purposes and in research settings, facilitating a deeper understanding of the inner workings of machine learning models.

One especially interesting use case of this visualization is investigating how the weights change when training data, or hidden layer structure is changed. This can help solve problems with over fitted models, unneeded layers that add computation time when training, or educational purposes.

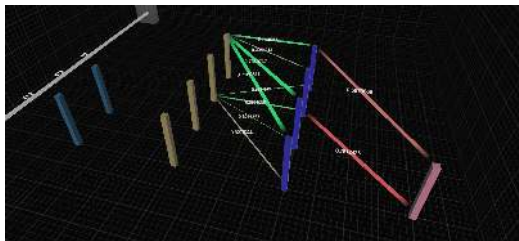
What sets this visualization approach apart is its use of the third dimension to reduce clutter, which is a common issue in traditional representations of machine learning models. By adding depth, the visualization becomes clearer and more navigable, allowing for an unobstructed view of the connections and interactions between layers. This decluttered interface enhances the ability to explore and interpret the complex relationships and dynamics within machine learning models.



(a) Traditional method of visualizing layers in 2D. Source: [?]



(b) Visualizing Layers in 3D.



(c) Focusing on specific layers & Weights using the Plot Viewport.

Figure 25: Comparison of Traditional Machine Learning Visualizations and what ImmerseDataXR offers.

4.2.3. Shader-Based Data Visualization

In an experimental approach to enhance data visualization capacity, a technique involving shaders and PNG textures was explored. In this method, data points were encoded within the RGB channels of a PNG image, where each pixel represented a distinct data point: the R, G, and B values corresponding

to the X, Y, and Z coordinates, respectively. A custom shader script was then employed to render these data points in a 3D space akin to a scatter plot, encapsulated within a cube. The cube's appearance, either transparent or black, could be adjusted to suit different viewing preferences (Figure ??).

This was achieved using a custom shader written in HLSL (High-Level Shading Language) [?]. The shader works as follows (Figure ??):

1. **Data Input:** The data is encoded into a PNG file, that is used as an input texture in the shader.
2. **Fragment Shader:** The fragment shader does all the work of rendering the plot inside of the cube.
 - (a) **Find position of fragment in World Space:** The shader initially converts and stores the position of the fragment from screen space to the position it has in world space.
 - (b) **Match fragment to a data point:** Goes through all data points in the Texture, until it finds a data point which matches the world space of the fragment.
 - (c) **Final Rendering:** If a data point is found that matches the current fragment, render the color of the data point. Otherwise paint black or transparent (Depends on setting).

However, this innovative method encountered significant challenges. One primary issue was the lack of interactivity with the rendered data, which limited its usefulness in dynamic data analysis scenarios. Additionally, the experiment faced performance limitations related to the size of the input image. Larger images, representing more extensive datasets, resulted in considerable slow-downs, making the visualization process impractical for larger datasets. Due to these challenges, particularly the lack of interactivity, the decision was made to discontinue this approach in favor of more efficient and interactive visualization methods.

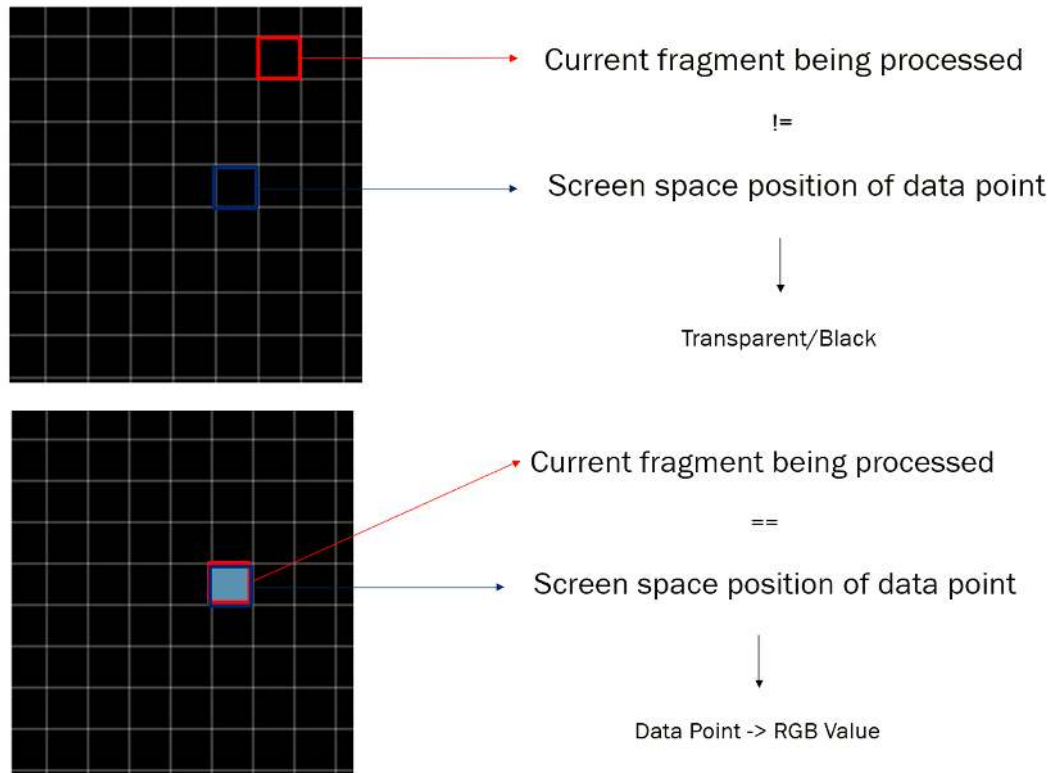
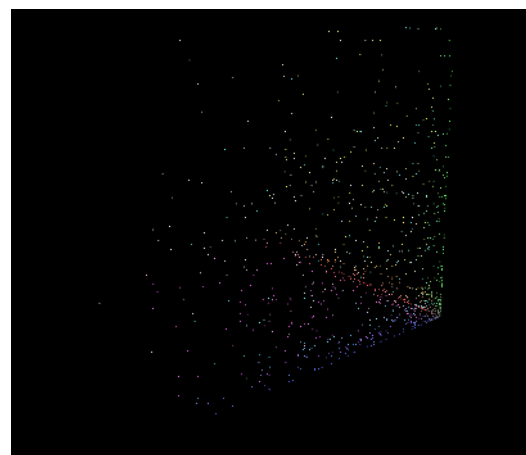


Figure 26: Flowchart of Shader to achieve shader based rendering.



(a) Shader plot in a 3D Environment.



(b) Shader plot in a black background.

Figure 27: Shader Plot in different environments.

5. Results

The cornerstone of our case study in data analysis through visualization was the worm lifespan experiment. This study aimed to observe the effects of various chemicals on the lifespan of worms, organized into groups based on chemical type and concentration. The experiment generated a dataset in the form of time series, where each time series depicts the daily deaths recorded per group. Our challenge was to effectively visualize these multiple time series concurrently, a task that traditional methods struggled with viewing multiple time-series.

By employing our 3D visualization approach within a XR environment, we were able to present these time series side by side, offering an intuitive and comprehensive view of the data. This enabled a clearer understanding of how different chemical treatments influenced worm lifespans and allowed for the early detection of experimental anomalies, which could indicate environmental interference or other issues affecting the results. The experiment's findings were instrumental in demonstrating the effectiveness and versatility of the ImmerseDataXR toolkit, not only in providing deeper insights into complex biological datasets but also in enhancing user accessibility for both educational and research purposes.

Crucial to the success of this experiment were the contributions of a professor of biology and their assistant, who provided invaluable feedback and insights. Their expertise in the field of biology greatly influenced our approach, as they shared their current methodologies and expressed their needs for more advanced visualization tools. Their perspectives on how biological data is typically analyzed and what would enhance their research were key factors in tailoring the visualizations made in the ImmerseDataXR toolkit to be more effective and user-friendly. Their involvement ensured that our visualization solutions were not only technologically advanced but also practically relevant and beneficial for real-world biological research.

5.1. Use Case Analysis

5.1.1. Technical Details of Data Provided

In both use cases, the data processing was handled using custom Python scripts developed specifically for each visualization. The input for these scripts was provided in the form of CSV (Comma Separated Values) files, which were

originally generated using the R programming language. These files contained detailed experiment data, structured to facilitate analysis and visualization in the experiments.

The format of the CSV files was tailored to capture the daily progress of each experiment. Each row in the files corresponded to a different day of the experiment. The columns were organized as follows: the first column contained the day number, representing the temporal aspect of the experiment. The second column listed the number of worm deaths recorded on that particular day. The third column contained the ID of the group, which was crucial for differentiating between the various groups treated with different chemicals. This structured format is common for these types of experiments and allowed for a clear representation of the data, ensuring that the subsequent analysis and visualization would accurately reflect the dynamics and outcomes of the experiments.

All files involved for all experiments can be found in Appendix ?? on page ??.



Figure 28: Pre-processing of Worm Lifespan Data.

5.1.2. Description of Use Cases

Comparison of Worm Treatments The first issue in our study involves the visualization and comparison of the effects of different chemicals on groups of worms. The challenge lies in representing these effects in a visually clean and uncluttered manner. Traditional 2D visualizations often fall short in this regard, as they tend to become cluttered and less interpretable, especially when dealing with multiple datasets. A clear and concise visualization is needed to effectively compare the impact of various chemical treatments across different worm groups, highlighting differences and similarities in their responses.

Detecting Outliers in a set of Time series The second problem is the detection of outlier time series within the data set. In the context of our experiment, an outlier would indicate a deviation from expected results, potentially due to an environmental variable affecting the experiment. Early detection of such outliers is crucial as it allows for timely intervention, potentially saving time and resources. The challenge is to identify these anomalies in a set of time series

data accurately and efficiently, ensuring that any compromised experiments are promptly addressed.

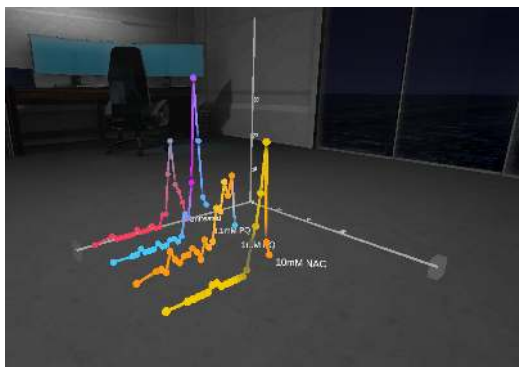
5.2. Comparison of Worm Treatments

In our approach to comparing different worm treatments, the key strategy was to place the time series of each treatment group next to each other. This side-by-side arrangement, coupled with specific color coding, was instrumental in differentiating between various experiments and their respective groups.

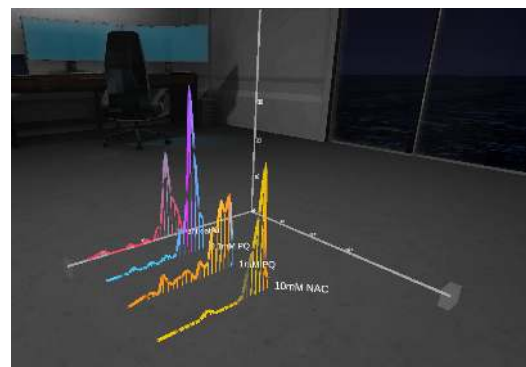
Each primary experiment consisted of several groups, each varying in treatments. To effectively distinguish these, we experimented with different color-coding schemes. These schemes aimed to enhance the clarity and understanding of how each treatment and its variables impacted the worm populations.

5.2.1. Using various methods for Time Series visualization

We explored various methods of visualizing the time series data to determine which was most effective for user comprehension (Figure ??). This included employing both bar plots and connected graphs as alternative visualization formats. The goal was to identify which method provided the clearest and most understandable representation of the data, thereby facilitating a more intuitive comparison of the treatment effects on the worm groups.



(a) Timeline Visualization using a Connected Plot.



(b) Timeline Visualization using a Bar Plot.

Figure 29: Different Timeline visualizations

5.2.2. Feedback and Evaluation of Visualization Techniques

During the evaluation of different 3D visualization methods for the experimental data, both the Connected Plot and the graph-based timeline were effective in distinctively separating and elucidating the various experiments. However, specific feedback from the collaborating professor and their student, revealed a marked preference for the connected plot based timeline's traditional appearance. This conventional format better aligned with their familiar analytical practices and expertise. Consequently, in response to their input, the graph-based method will be implemented in future, more intricate experiments. This adaptation ensures that the visualization approach is in harmony with the expert users' preferences, thereby enhancing both clarity and ease of interpretation in complex data analysis.

5.3. Detecting Outliers in a Set of Experiments

The research addressed the complex challenge of detecting outliers across an extensive set of time series data. This task diverged from the conventional practice of finding outliers within a single time series.

The data set encompassed results from four different primary experiments, each consisting of four groups. Each of these experiments and their respective groups shared similar attributes in their experimental conditions, making them suitable for an integrated analysis. To handle this complexity, all the data from these multiple experiments was amalgamated into a single comprehensive data set. The study then applied the following methods to this aggregated data to systematically identify outliers.

5.3.1. Using K-Means Clustering

The initial approach for detecting outliers in the set of time series employed k-means clustering, systematically applied to each series:

1. **Training the Model:** A K-Means Clustering model with $K = 3$ using the `sklearn.cluster.KMeans` Python library is trained using the entire data set.
2. **Assessment of Fit:** The fit of each examined time series within the model's clusters is evaluated. The appropriate cluster to which the time series belonged is identified.
3. **Distance Calculation and Scoring:** For each time series within the same

cluster, the distance from the examined series is calculated. An average of these distances is computed, representing the outlier score. A larger score indicates a poorer fit and a higher likelihood of the series being an outlier.

4. Iterative Approach: This procedure is repeated iteratively for every time series in the data set, with each iteration involving a retraining of the model to ensure a comprehensive evaluation.

A key aspect of this method was its use of clustering to preliminary filter out potential outliers in the data set. This approach ensured that each time series was compared only with those similar to it, effectively isolating series that markedly deviated from the norm. Such a refined method facilitated a more precise assessment, focusing the analysis on series that generally adhered to the trends within the clusters.

Through this methodical approach, the study effectively ascertained the degree of deviation or conformity of each time series with the primary clusters. Consequently, it was possible to accurately identify the series that significantly diverged from common patterns, categorizing them as outliers.

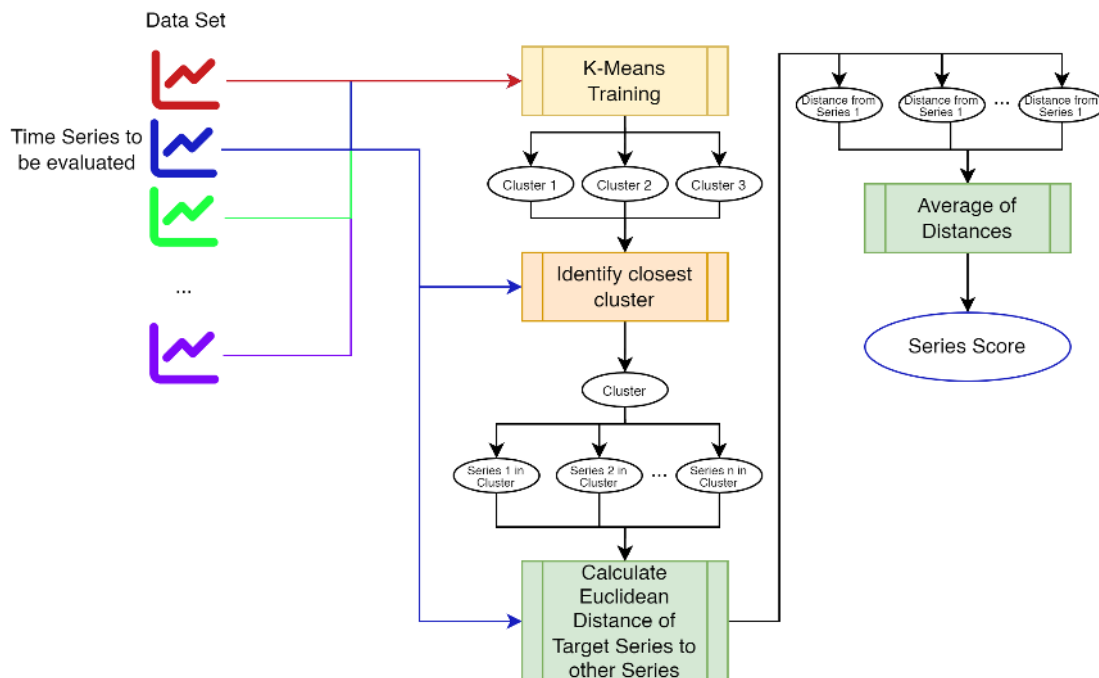


Figure 30: K-Means clustering method for detecting outliers in one Time Series.

Following the calculation of outlier scores for each time series, an essential step of normalization was implemented. This process adjusted the scores to fall within a range from 0 to 1. Subsequently, a color-coding scheme was applied to each time series within the visualization, enhancing the interpretability of the data. The coloring indicated that a time series colored closer to red in-

licated a higher degree of outlier status. In essence, the redder a time series appeared in the visualization, the more it deviated from the norm, marking it as a significant outlier. This visual representation provided an intuitive and immediate understanding of the data, allowing users to quickly identify and focus on the most divergent series within the data set.

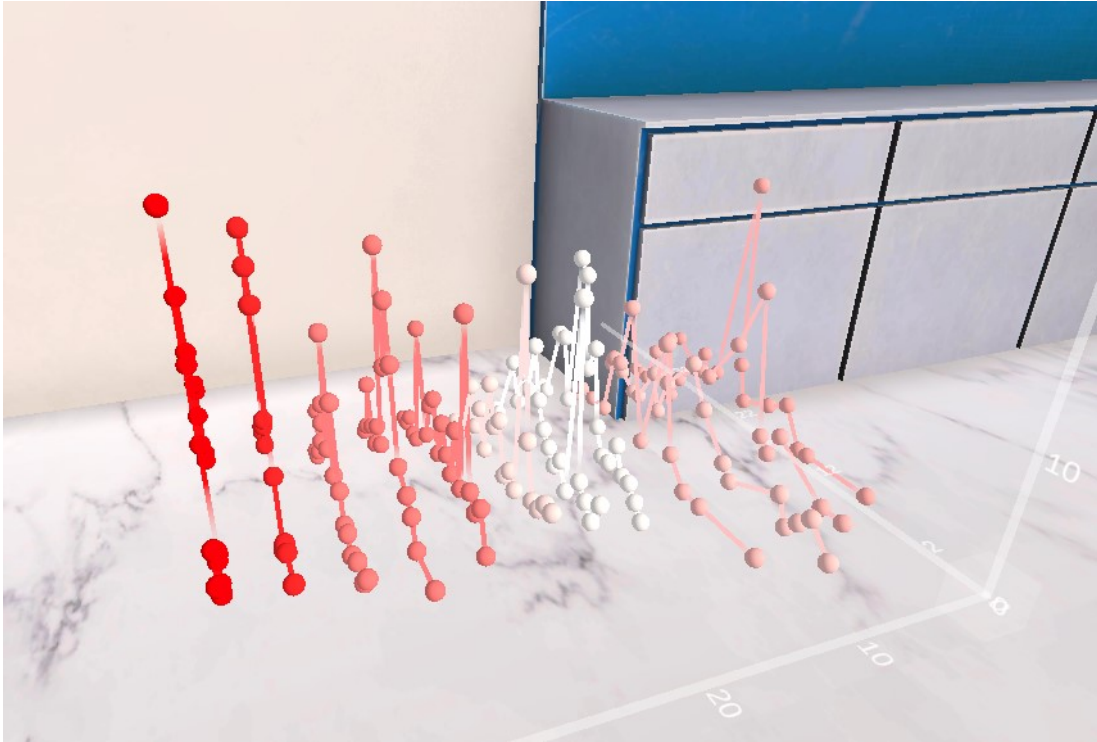


Figure 31: Plots Colored based on their outlier scores, only using three experiments.

5.3.2. Detecting Outliers Before Experiment Completion

While the initial approach marked a significant step forward, the real challenge lies in identifying problematic time series before their completion. The goal was to detect anomalies in ongoing experiments, rather than analyzing complete time series post-experiment.

To validate this approach, cooperation with the biology experts was sought. The professor and their student provided datasets from experiments that were known to have either proceeded correctly or encountered issues. This data served as a critical test bed for the modified algorithm.

The adaptation made to the algorithm involved analyzing time series up to a specific day, rather than considering them in their entirety. By evaluating all series up to a given date, including the target series, the algorithm was able to assign daily outlier scores. This modification allowed for a dynamic, day-by-day analysis, coloring each day based on the relative normalcy of the

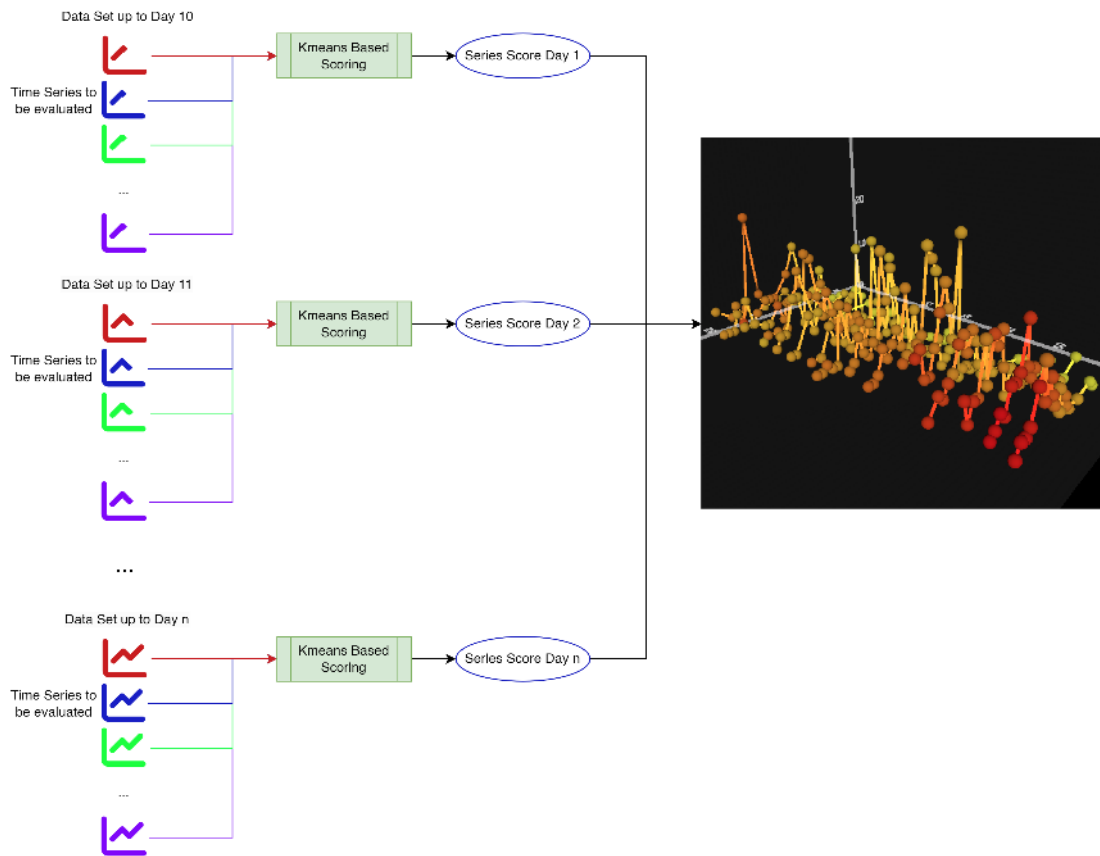


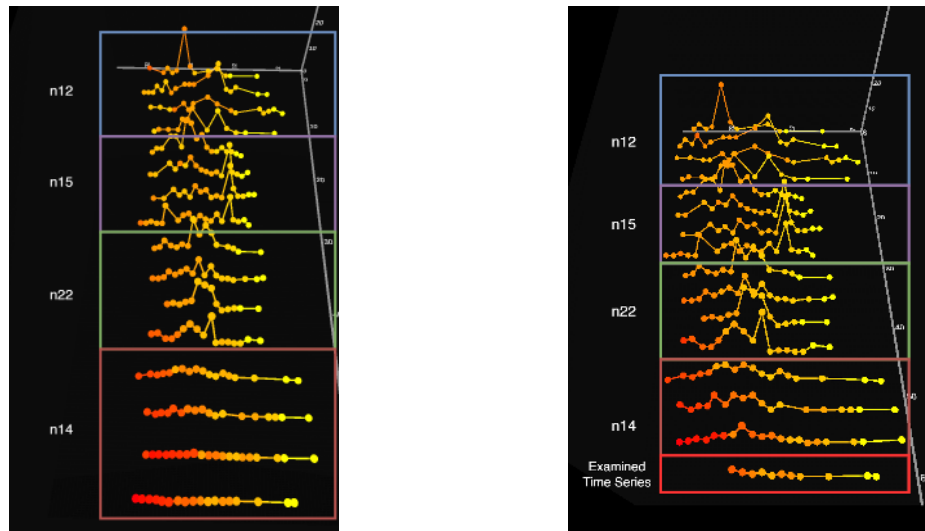
Figure 32: K-Means clustering method for detecting outliers in one Time Series, with changing weights per day.

ongoing experiment compared to others. Following the professor's advice, the algorithm was adapted to start analyzing the time series from day 10, as this was identified as the critical point where deviations typically began to manifest.

Such an approach offered the crucial advantage of real-time monitoring. As soon as the data began deviating anomalously from other experiments, it would be visually evident in the color-coded time series. This immediate feedback mechanism enables researchers to identify issues as they occur, rather than retrospectively analyzing the data after the experiment's conclusion. Consequently, this method significantly enhances the ability to take timely corrective actions, potentially saving both time and resources.

We obtained data from four distinct experiments, labeled as n12, n15, n22, and n14. Among these, the n14 dataset was of particular interest as an environmental variable had adversely affected the results. This data set served as a critical test case for the system's ability to flag outliers.

In the analysis, the time series corresponding to the n14 experiment were prominently flagged by the system as outliers. As illustrated in the following figure, the four groups that are contained in the n14 experiment, are clearly



(a) Detected an issue with the last experiment.

(b) Detected issues with the last, unfinished group.

Figure 33: K-Means clustering method for detecting outliers per day.

distinguishable as outliers (Figure ??). This visual clarity in the system's flagging process effectively highlighted the anomalous nature of the n14 data.

Further testing involved removing some days from the data of the last group in the n14 experiment. The objective was to assess whether the system could still successfully identify the incomplete time series as outliers. The results were in line with expectations: the outlier detection mechanism functioned effectively, successfully identifying the n14 group as outliers despite the incomplete time series data. (Figure ??) This outcome demonstrated the robustness and reliability of the system in identifying outliers, even in cases of partial data availability.

5.3.3. Using the Outlier Rank Algorithm

Using K-Means or similar clustering techniques can be time consuming especially as data sets get larger or more complex. For this reason, a novel algorithm was developed to detect outliers in time series by leveraging measures of central tendency and dispersion. This approach involves a detailed analysis of each time series within the data set. The algorithm works as follows (Figure ??):

1. Obtaining key Statistical Measures from the Data Set: The algorithm first calculates key statistical measures for each time series, including the minimum (min), maximum (max), lower quartile (Q1), upper quartile (Q3), and mean. These measures provide a comprehensive overview of the data distribution

within each series.

2. Extracting Interquartile Range from statistical measures: The algorithm then, for each statistical measure, finds the upper (Q3) and lower quartiles (Q1) for the values from each data set. It also calculates the interquartile range (IQR).

3. Detecting outliers in statistical measures: These values are then used to determine if the measure from each data set is an outlier. If the measure lies outside of the range, it is considered an outlier. The α value is used as a sensitivity value. Greater α values allow for less strict detection of outliers, but lower values may lead to more false positives.

$$\text{Lower Bound} = \text{LowerQuartile} - \alpha \times \text{IQR}$$

$$\text{Upper Bound} = \text{UpperQuartile} + \alpha \times \text{IQR}$$

4. Determining the Outlier Rank: Finally, the number of outliers in each time series within the data set is added up, resulting in an outlier rank. The higher the rank, the more of an outlier the time series is.

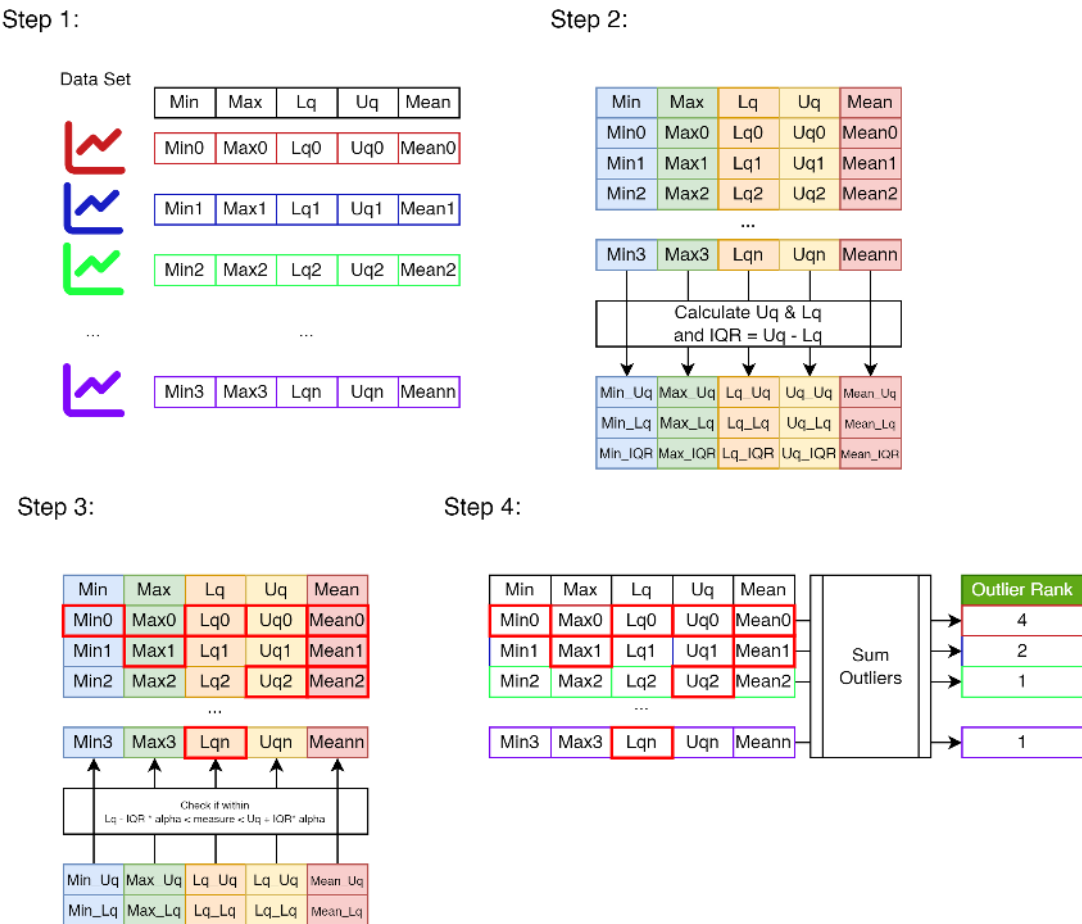
After the rank is determined, the values are normalized and each node is colored more red the higher the rank is, similarly to the K-means method.

With an α value of 0.2, we have similar results to the ones provided by the K-means clustering method. However, we also have some positives on groups that were considered correct. This is an interesting phenomenon however, as upon closer inspection these groups indeed do not look similar to the others (Figure ??).

5.3.4. Using a Combination of K-Means and the Outlier Rank Algorithm

To enhance the accuracy and robustness of outlier detection, a synergistic approach was adopted, combining the strengths of both k-means clustering and the Outlier Rank algorithm based on measures of central tendency. This integrated method leverages the unique capabilities of each technique to provide a more comprehensive analysis. The process involves assigning weights to the ranks obtained from both the k-means clustering and the Outlier Rank algorithm.

$$\text{Score} = \text{KMeans_Score} \times w_1 + \text{OutlierRank} \times w_2$$



Step 3:

Min

Max

Lq

Uq

Mean

Min0

Max0

Lq0

Uq0

Mean0

Min1

Max1

Lq1

Uq1

Mean1

Min2

Max2

Lq2

Uq2

Mean2

...

Min3

Max3

Lqn

Uqn

Meann

Check if within
 $Lq - IQR * \alpha < measure < Uq + IQR * \alpha$

Min

Uq

Max

Uq

Lq

Uq

Lq

Uq

Mean

Uq

Min_Lq

Max_Lq

Lq_Lq

Uq_Lq

Mean_Lq

Step 4:

Min

Max

Lq

Uq

Mean

Min0

Max0

Lq0

Uq0

Mean0

Min1

Max1

Lq1

Uq1

Mean1

Min2

Max2

Lq2

Uq2

Mean2

...

Min3

Max3

Lqn

Uqn

Meann

Sum
Outliers

Outlier Rank

4

2

1

1

Figure 34: Novel Algorithm Flow of Execution.

By weighting these ranks, the method balances the insights gained from each approach, effectively amalgamating the different perspectives they offer on the data. For various combinations and weights of the two methods, the highlighting changes and focuses on different parts of the time series (Figure ??).

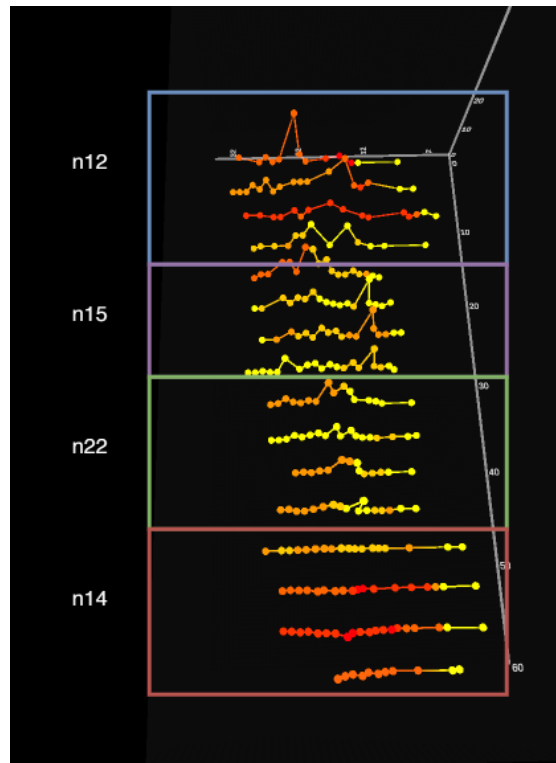
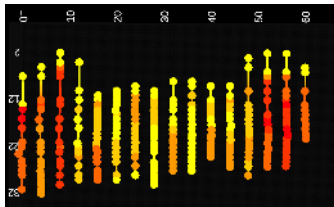
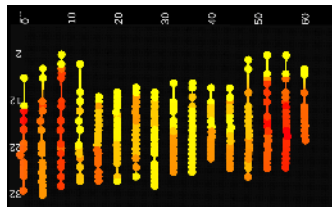


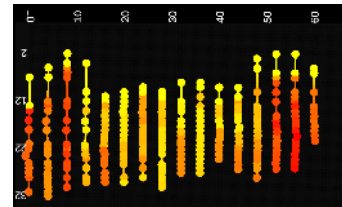
Figure 35: Resulting plot from using the Outlier Rank Algorithm with $\alpha = 0.2$.



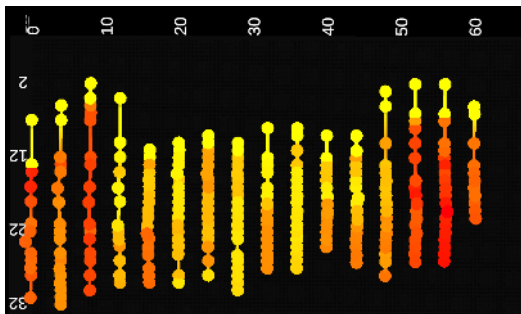
(a) 0% KNN, 100% Outlier Rank Algorithm



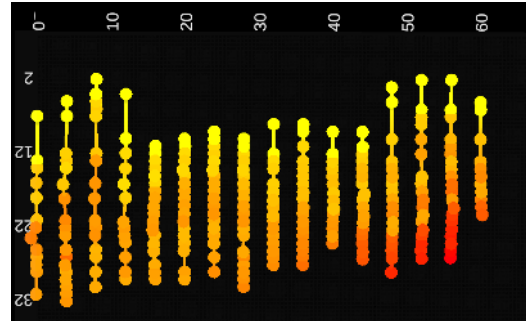
(b) 25% KNN, 75% Outlier Rank Algorithm



(c) 50% KNN, 50% Outlier Rank Algorithm



(d) 75% KNN, 25% Outlier Rank Algorithm



(e) 100% KNN, 0% Outlier Rank Algorithm

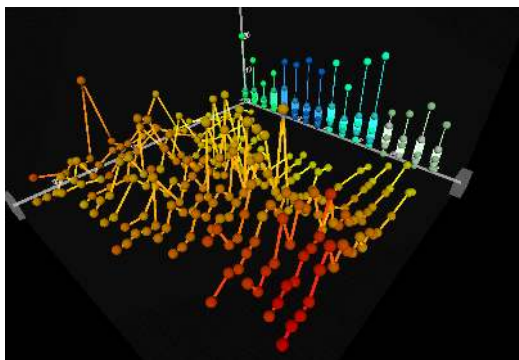
Figure 36: Combined Values from KNN Clustering and Outlier Rank algorithm at different weight combinations.

5.3.5. Using Visualized Statistical Measures

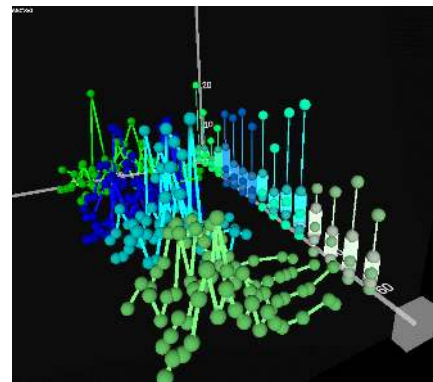
In conjunction with algorithmic calculations, this study also employs visualizations of key statistical measures – including upper quartile (UQ), lower quartile (LQ), minimum (min), and maximum (max) – in the form of box plots. These box plots serve as a powerful tool to condense the complex time series data into more digestible 2D visual summaries.

The top whisker of the box plot represents the maximum value of the time series, the bottom whisker the minimum, the start of the box the Upper Quartile, the End of the Box the Lower Quartile, and the point inside the box the median value of the time series.

These visual summaries are designed to complement the main plots, whether they include algorithm-highlighted (Figure ??) outliers or not (Figure ??). By presenting box plots alongside the primary visualizations, users are afforded the opportunity to draw their conclusions independently, relying on their analysis and interpretation of the visual data. This aspect is particularly advantageous, as it allows for a more intuitive understanding of the data, unmediated by algorithmic influence.



(a) Highlighted with scores obtained via K-Means.



(b) Colored based on which experiment each group belongs to.

Figure 37: Box Plot with Time Series.

The incorporation of box plots in the visualization process offers a significant benefit. It presents the statistical distribution of each time series in a format that is both visually appealing and informative. This approach is particularly effective in a 3D environment, where traditional 2D plots might fall short in conveying the depth and nuances of the data. By enabling users to see the spread and central tendency of each time series at a glance, box plots provide a valuable perspective that enhances the overall comprehension of the dataset.

This methodology not only leverages the clarity and simplicity of 2D visualiza-

tions but also capitalizes on the immersive and interactive nature of 3D environments. It empowers users to engage with the data on multiple levels, combining the straightforwardness of box plots with the depth of 3D visualization to facilitate a more rounded and thorough data analysis experience.

5.4. Data Processing Workflow

The workflow for data processing in this project was efficiently designed to utilize just two Python scripts, streamlining the process from raw data to final visualization.

Statistical Measure Calculation The first script takes the original CSV file containing the experimental data as input. Its primary function is to calculate key statistical measures for each time series, which are essential for the subsequent visualization steps. Once calculated, these statistical measures are saved into a new CSV file, which serves as an intermediary output in the data processing pipeline.

Generation of Visualization Input Files The second script in the workflow utilizes both the original CSV file and the newly created CSV file containing the statistical measures. Its role is to generate three distinct input files tailored for the ImmerseDataXR visualization toolkit.

1. **Prefab Plot Input File:** This file is designed to visualize the time series in a prefab plot format, providing a straightforward representation of the data.
2. **Connected Plot for Time Series Visualization:** The second output file is formatted for a connected plot visualization. It presents the time series without highlighting outliers but includes the integrated box plots, offering a comprehensive view of the data distribution.
3. **Connected Plot Highlighting Outliers:** The third file focuses on visualizing outliers along with the box plots in a connected plot format. This version emphasizes the detected outliers, making it easier to identify and analyze deviations in the data.

Through this two-step Python scripting process, the raw data is transformed into visualization-ready formats. Each output file caters to different aspects of the data, from basic time series representation to more advanced visualizations focusing on outliers and statistical distributions, thereby providing a versatile toolkit for comprehensive data analysis.

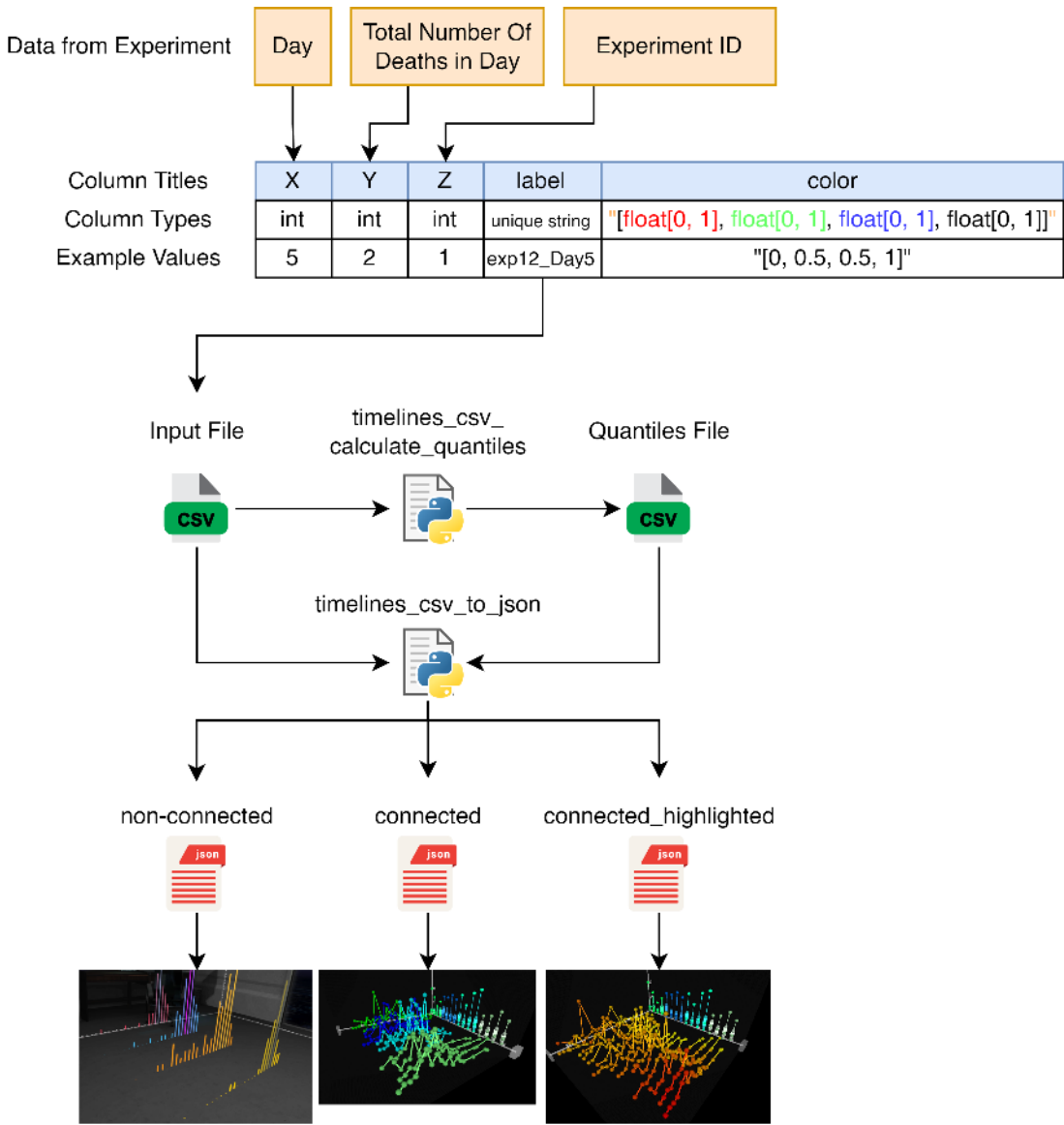


Figure 38: Usage of Use Case Python scripts to generate visualizations for ImmerseDataXR.

6. Discussion & Future Work

This section delves into a comprehensive discussion of the findings from our major use case, with a specific focus on how the ImmerseDataXR visualization toolkit has enhanced the analysis and interpretation of biological data. We will explore the implications of these findings, both in terms of technological innovation and practical application in the field of biology. Through those findings, we will also evaluate the effectiveness of the ImmerseDataXR toolkit for creating Immersive and Interactive Data Visualizations.

6.1. Key Findings in Biological Data Use Case

This subsection delves into the notable discoveries made by biologists through the application of 3D visualization tools in their data analysis.

System Flexibility A significant aspect of this research was the flexibility of the visualization toolkit, which facilitated numerous iterations and modifications with minimal effort. This adaptability proved crucial in the context of biological data analysis, as different visualization approaches can yield varied results. The ability for rapid prototyping and easy adaptation of the visualizations to meet specific analytical needs was a key benefit. It allowed biologists to experiment with various visualization formats to identify the most effective methods for their data interpretation. This agility in visualization development is particularly valuable in fields where data and research objectives can vary widely, necessitating a tool that can quickly adjust to new requirements and perspectives.

Enhanced Comparative Analysis in Treatment Studies A pivotal finding in the treatment comparison use case was the capability to concurrently visualize a greater number of experiments than what traditional 2D plots allow. This feature facilitated the side-by-side comparison of multiple experiments, providing a comprehensive view that enhanced the understanding of different treatments' relative effectiveness. The 3D visualizations offered a holistic perspective, aiding biologists in drawing more accurate and nuanced conclusions about treatment efficacy.

Innovative Approach to Outlier Detection In outlier detection, the visual scoring and color-coded time series representation were particularly impactful. This novel method provided dynamic and evolving insights into the progression of experiments. The inclusion of box plots and highlighted areas in the visu-

alization introduced a fresh perspective to data analysis. Moving away from traditional, sometimes obscure scoring methods or subjective evaluations, the immersive 3D visualizations offered a tangible and user-friendly means for assessing experimental outcomes. This was especially beneficial for less experienced biologists or users from other disciplines, who could now identify potential issues in experiments without the need for deep expertise or reliance on complex scoring algorithms. This accessibility to intuitive data evaluation tools democratizes the process of scientific analysis, making it possible for a broader range of users to make informed decisions and observations in biological research.

These key findings emphasize the transformative role of immersive 3D visualizations in biological data analysis. They highlight the tools' potential not only to augment existing practices but also to pioneer novel methods for data exploration and interpretation, catering to experts and novices alike.

6.2. Advantages Over Traditional Visualization Methods

Enhanced Information Conveyance A primary advantage of these immersive visualization tools is their superior capability in conveying complex information efficiently. Compared to standard 2D plots, these tools provide a dynamic and multi-layered perspective. They empower users to understand the nuances of multi-dimensional data, which often remain obscured in two-dimensional representations. The rich depth and detail achievable in an immersive 3D environment surpass the visual limitations of flat, two-dimensional plots, offering a more thorough comprehension of complex datasets.

Efficiency in Data Interpretation The tools significantly enhance the efficiency of data analysis by presenting more information in a shorter time frame. Users can observe multiple data dimensions simultaneously, not just in terms of spatial arrangement but also through additional layers such as color coding and other visual cues (Explored with the Combined plots of time-series and Box Plots). This multi-dimensional representation accelerates the process of data interpretation, making it possible to extract insights more quickly and effectively than traditional methods.

Improved 3D Interaction and Clarity Another notable benefit is the heightened interactivity and clarity these tools offer compared to traditional 3D plots on a 2D screen. Standard 3D visualizations often suffer from overlapping data points and obscured details when confined to a flat display. Conversely, im-

mersive XR environments provide an authentic 3D experience, enabling users to interact with and explore data in a spatially precise and intuitive manner. This enhanced interactivity not only improves the clarity of data representation but also deepens the user's engagement with the data, making the analysis process more intuitive and user-friendly.

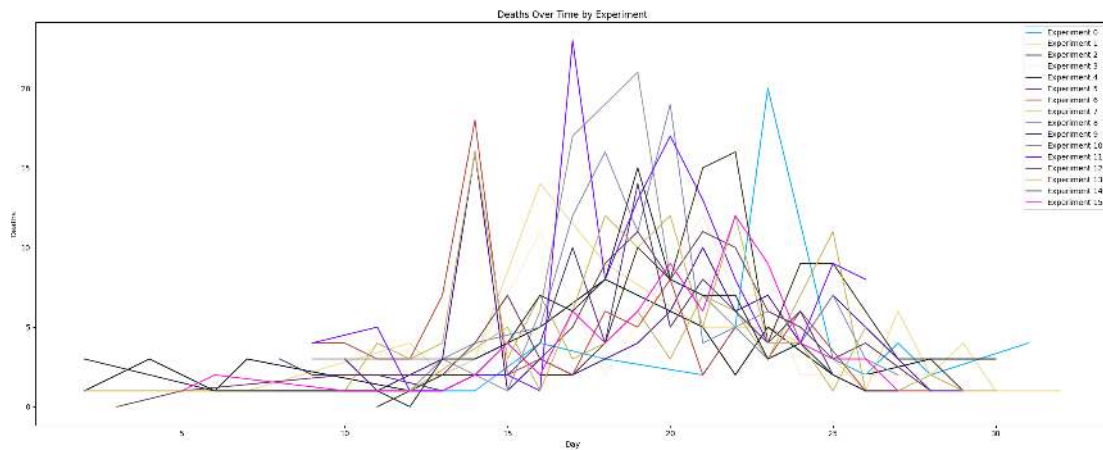
Enhanced Collaboration in XR The integration of XR in data visualization significantly enriches the collaborative experience. It fosters a sense of shared space, enhancing interactions and making them more intuitive. This aspect is particularly beneficial for mentorship and education, where complex ideas can be conveyed effectively in a three-dimensional, interactive environment. Real-time collaboration in VR also streamlines teamwork, allowing for immediate feedback and joint problem-solving. Moreover, the immersive nature of VR strengthens the emotional connection and engagement with both the data and fellow collaborators, leading to a more invested and enthusiastic approach to data analysis.

In conclusion, the immersive visualization tools represent a significant leap forward in data presentation and analysis. Their ability to encapsulate complex information in an interactive, three-dimensional environment not only boosts the accuracy of data interpretation but also improves the overall efficiency of the analytical process. This marks a notable advancement in the realm of data visualization.

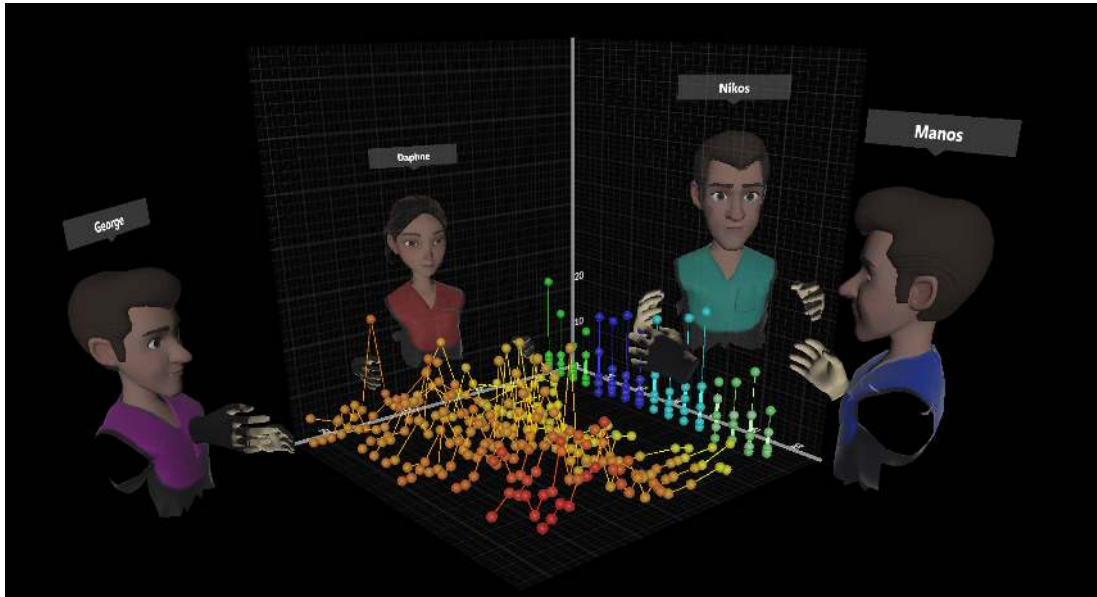
6.3. Challenges and Constraints

This subsection addresses the primary challenges and constraints encountered in the application of XR for biological data analysis.

Accessibility and Familiarity with XR One significant limitation noted was the accessibility and general familiarity with XR technology. The cost factor associated with obtaining specialized hardware for XR setups posed a barrier, making it less feasible for widespread adoption in some settings. Additionally, there was a noticeable preference for traditional desktop interfaces, particularly among more seasoned professionals, who are less accustomed and open to using new technologies. While the younger generation showed enthusiasm for the interactive nature of XR, this sentiment was not universally shared, indicating a generational divide in the acceptance and preference for XR-based analysis tools.



(a) 2D



(b) 3D

Figure 39: Experiments n12, n14, n15, n22 using different visualization methods.

Performance Issues with Large Data sets Another challenge faced was the handling of large datasets in XR. As the complexity and size of the datasets increased, performance issues such as lower frame rates and higher loading times became more pronounced.

Complexity in Visualization The study also found that at times, the plots within the XR environment could become overly complex and congested, necessitating additional explanations for users. This complexity, while indicative of the depth of analysis possible with XR, also highlighted the need to be careful as to how much data or different visualizations are displayed at any time. Simplifying the visual presentation without losing the richness of data is a key

area for future development, ensuring that the insights derived from the visualizations are easily comprehensible to all users, regardless of their expertise level.

These challenges, while highlighting areas for improvement, also underscore the evolving nature of immersive visualization technology in biological data analysis. They provide valuable insights into the current state of XR in scientific research and pave the way for future enhancements that could mitigate these constraints.

6.4. Implications for Biological Data Analysis

This subsection explores the transformative implications of the ImmerseDataXR toolkit for the field of biological data analysis, particularly in light of generational shifts in technology adoption and the integration of novel analytical methodologies.

Generational Shift in Technology Adoption A notable observation from our study is the generational divide in the adoption of immersive visualization technologies. Younger generations within the biological research community appear more inclined to embrace advanced tools like XR for real-time data exploration. This trend suggests a shift towards more dynamic and interactive forms of data analysis, moving away from traditional methods. The integration of XR, especially in ongoing experiments, offers a more immediate and immersive understanding of data, which could significantly enhance research efficiency and outcomes.

Updating Traditional Data Analysis Methods The field of biology has traditionally relied on conventional forms of data representation, such as 2D plots, and often utilized analytical techniques that have remained unchanged for decades. Our research introduces a paradigm shift, not only by employing state-of-the-art visualization techniques but also by incorporating advanced mathematical and analytical concepts previously underutilized in biological data analysis. This integration represents a significant step forward, aligning biological research with contemporary practices prevalent in other scientific disciplines.

Broadening the Analytical Horizon By introducing these modern analytical approaches and visualization techniques, we are broadening the horizon for biological data analysis. The toolkit offers a new perspective, enabling biologists

to uncover insights that were previously inaccessible due to the limitations of traditional methods. This advancement is not merely a technological upgrade; it represents a fundamental change in how biological data is interpreted and understood. The potential for these tools to drive innovation and discovery in biology is immense, particularly as they become more integrated into the standard research workflow.

6.5. Reflection on Research Objectives

Development of a Versatile Visualization Toolkit A primary goal of this research was to develop a highly flexible toolkit for XR visualizations, suitable for various programming and data analysis needs. This objective was substantially met. We successfully demonstrated that experienced users, with minimal data preprocessing skills, could adapt their datasets for use with our system. This flexibility was further highlighted by the creation of a wide array of visualizations using the toolkit's foundational elements – the Prefab Plot and Connected Plot. These two plot types, through creative application, were adapted to develop numerous distinct visualizations, showcasing the toolkit's adaptability and potential for diverse data representation.

Enabling Detailed Data Insights The toolkit proved to be a robust platform for gaining in-depth insights into complex datasets. It allowed for analyses and visualizations that went beyond the capabilities of traditional methods. This success aligns with our objective of enhancing data comprehension and interpretation through immersive and interactive visualizations.

Diverse Visualization Applications A significant achievement of this project was the development of various types of visualizations, all inherently supported by the toolkit. Through the flexible use of the Prefab Plot and Connected Plot, we created a range of visualizations, each showcasing the toolkit's versatility. From simple scatter plot to various time series, the toolkit demonstrated its capacity to cater to a wide spectrum of visualization requirements, far exceeding the conventional usage scenarios.

Challenges in User Empowerment Despite these successes, one aspect where the project fell short was in enabling biologists to independently create and customize their visualization solutions. Time constraints limited our ability to fully develop a user-driven approach for solution creation. Although we engaged in collaborative visualization development with biologists, empower-

ing them to independently use and adapt the toolkit for their specific research needs remains an area for future development. Addressing this will be critical for broadening the toolkit's application and enhancing its impact within the biological research community.

In summing up, the ImmerseDataXR toolkit has achieved several key objectives, particularly in creating a flexible platform for immersive visualizations and enabling profound insights into complex data sets. However, ongoing work is needed to empower users, especially in the biological sciences, to fully leverage the toolkit's capabilities in their independent research endeavors.

6.6. Future Work

This subsection outlines the potential avenues for future research and development based on the experiences and findings from the ImmerseDataXR project.

Performance Enhancements A primary focus for future development is enhancing the performance of the ImmerseDataXR toolkit, particularly when handling large datasets. Performance optimization will be crucial to ensure smooth and efficient operation, especially in XR environments where computational demands can be significant. Research into more efficient data processing techniques and graphics rendering optimizations will be undertaken to improve the overall user experience and enable the handling of more complex and voluminous datasets.

Expansion of Statistical Analysis Features A crucial area for future development is the enhancement of the Novel Algorithm's used for detecting outliers analytical capabilities, particularly by incorporating additional statistical measures into the novel algorithm used for outlier detection. The integration of these new measures is intended to refine the algorithm's precision, thereby increasing its accuracy and reducing the occurrence of false positives, which were noted during the initial research phase. By broadening the range of statistical measures within the algorithm, it can offer more robust and reliable analysis, making it an even more valuable asset for complex data examinations.

Adapting Visualization for Broader Platforms Recognizing the limitations in accessibility due to the focus on XR platforms, future work will also include adapting the visualization methods for use beyond immersive environments. Developing versions of the visualization tools that can be effectively used on

desktop or mobile platforms will significantly broaden the toolkit's user base. This adaptation will focus on retaining the interactive and three-dimensional aspects of the visualizations while ensuring they are usable on more widely accessible platforms.

Enhanced User Empowerment Continuing from the feedback received, a key area of future work will be to empower users, particularly those in the biological sciences, to independently create and customize their visualizations. Efforts will be made to streamline the user interface and simplify the data integration process, making it more accessible for users with varying levels of technical expertise. This endeavor will include the development of more intuitive tools and resources to guide users through the process of creating and adapting visualizations to fit their specific research needs.

User studies and Evaluation In this research, our primary focus was on addressing the visualization challenges within biological data analysis. While we have made significant strides in developing an immersive visualization toolkit, we recognize the importance of user studies and evaluations in further refining our tool's capabilities. Future work will include comprehensive user studies to assess the toolkit's usability and effectiveness from both user and developer perspectives. These studies will aim to identify areas for improvement in user experience, interface design, and functionality enhancements. Through this iterative process of development and evaluation, we aim to advance the toolkit's utility and user-friendliness, ensuring it meets the evolving needs of its users in scientific research and beyond.

In conclusion, the future work for the ImmerseDataXR toolkit will focus on enhancing its performance, expanding its analytical capabilities, broadening its accessibility beyond XR platforms, and empowering users to take full advantage of its features for their individual research purposes. These developments will ensure that the toolkit remains a cutting-edge resource in the field of data visualization.

7. Conclusion

Throughout this research, we have embarked on a comprehensive exploration of immersive analytics, probing its capabilities, development process, and real-world applicability, particularly in the realm of biological data visualization. We delved into the theoretical underpinnings of immersive analytics, examining how it has evolved and the distinctive benefits it offers. A significant focus of this study was on demonstrating how immersive analytics, when combined with other domains of research, can facilitate the creation of impactful and insightful visualizations.

One of the key findings of our research is the effectiveness of visual analytics in various fields and how the integration of Virtual Reality (VR) and Augmented Reality (AR) technologies can substantially enhance data comprehension. The development of the ImmerseDataXR toolkit, the cornerstone of our research, showcases how added interactivity can deepen the understanding of complex datasets. This interactivity, a feature less emphasized in other state-of-the-art solutions, proved to be vital in engaging users with their data more intuitively and effectively.

Our exploration also ventured into various methods of interacting with data. While not all approaches yielded successful outcomes, their examination was crucial in understanding the boundaries and possibilities within this domain. It is through these trials that we were able to refine our toolkit, adapting it to be more aligned with the practical needs of users, especially in the context of biological research.

As we conclude this study, it's clear that there remains a vast landscape of possibilities for future work. The journey of enhancing and expanding the capabilities of the ImmerseDataXR toolkit is far from over. We anticipate that this toolkit will open new avenues for researchers and professionals across various disciplines, enabling them to transform their data analysis processes. The potential for what can be achieved with this toolkit is expansive, and we eagerly look forward to seeing the innovative applications and developments that emerge from its utilization.

In summary, this research represents a significant step forward in the field of immersive analytics. It highlights the transformative power of VR and AR in data visualization and underscores the importance of interactivity in understanding complex data. As we move forward, the lessons learned and the groundwork laid by this research will undoubtedly contribute to the continued evolution of immersive data analysis, shaping the future of how we interact with and understand the vast expanse of data that defines our world.

A Appendix: Configuring Prefab and Connected Plots in Unity

1.1. Plot Viewport in the Unity Hierarchy

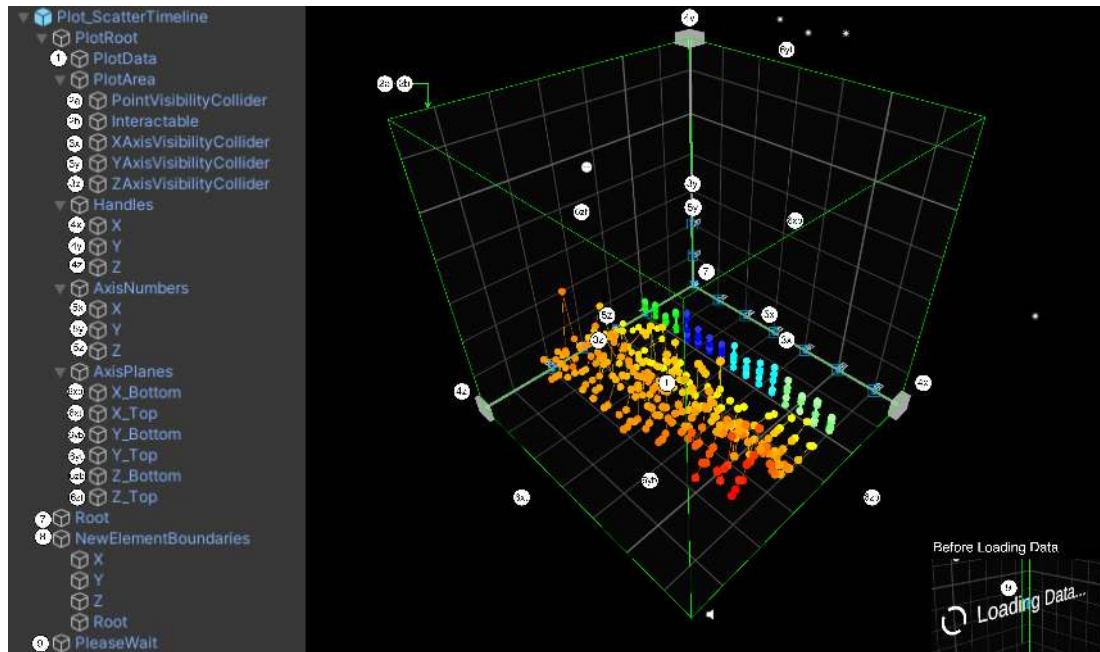


Figure 40: Plot View in the Unity Hierarchy.

The following outlines the structure of a "Plot Viewport" in the Unity Hierarchy, specifically designed for the use case of "ImmerseDataXR".

Before we go over all the points in Figure ??, we must define one of the more important technical parts of this system, the **Visibility Collider**.

The Visibility collider employs custom collision logic to detect and enable or disable the visibility of any pre-defined GameObjects that collide with it. This is the basis of the way many things in the Plot Viewport work, including the axis customization and axis numbers.

1. **Plot Data:** This is where the actual data being visualized is located within the Unity scene. Each point is a child GameObject to this Parent GameObject. Affected by the Display Collider.

2. Plot Management Colliders:

- (a) *Display Collider:* Dictates the data that is visible within the viewport, focusing the view this collider changes its size depending on the size of the axes.

- (b) *Interaction Collider*: Used for manipulating the data display, enabling moving and scaling based on user interactions. Its transform is identical to the Display Collider.
- 3. **Visibility Collider for Axis Numbers**: Controls the visibility of axis numbers.
- 4. **Handles for Changing Axis Size**: Interactable objects that enable users to adjust the size or scale of the axes for a customizable data view.
- 5. **Axis Numbers**: Numerical labels on the axes, providing scale and reference points for data interpretation. Affected by the Visibility Collider for Axis Numbers.
- 6. **Axis Planes (Grids)**: Structural background grid lines or planes that aid in spatial orientation and measurement. Simply a plane with a custom texture, that is parametrized by the location and scale of the Data Location (Point (1)).
- 7. **Handle for Moving the Entire Plot**: Interactable object for repositioning the entire plot within the virtual environment.
- 8. **Axis Handles in Data Dive Mode**: Where the Points (4x, 4y, 4z) and the Point (7) will be when a user enters data dive mode. Essentially defines the size and shape of the room the user is in.
- 9. **'Please Wait' Text During Data Loading**: A user interface element that displays a message while data is being loaded into the viewport.

Any plots created with ImmerseDataXR must follow this exact pattern, order and naming to work. Any deviation might cause Null Reference errors or unwanted behaviour.

1.2. Plot Editor UIs in Unity Inspector

Note

All prefabs are customizable, but they must retain a hierarchy similar to the included prefab.

1.2.1. Prefab Plot

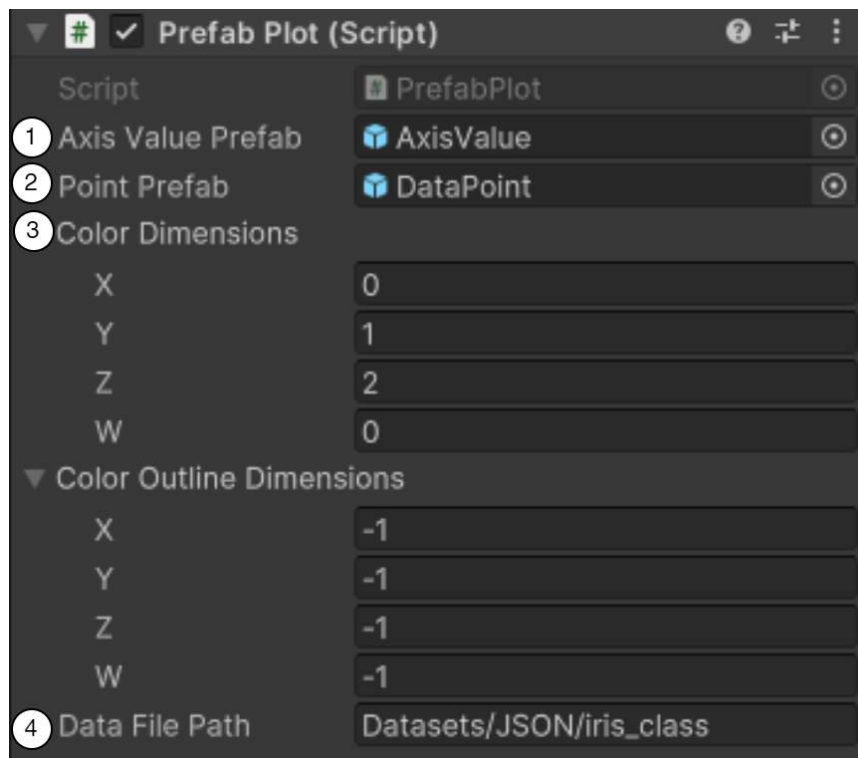


Figure 41: Prefab plot in Unity Inspector.

The key components of the prefab are as follows:

1. **Axis Value Prefab:** The prefab that is used for the axis values.
2. **Data Point Prefab:** The prefab that represents individual data points in the visualization.
3. **Color Dimensions:** This allows the selection of any column from the input file to determine the color of the data points, or -1 to use the color defined in the input file. This enables the functionality of the Customization plot discussed in Subsection ??.
4. **Input File:** The source file from which the data for the prefab is drawn.

1.2.2. Connected Plot

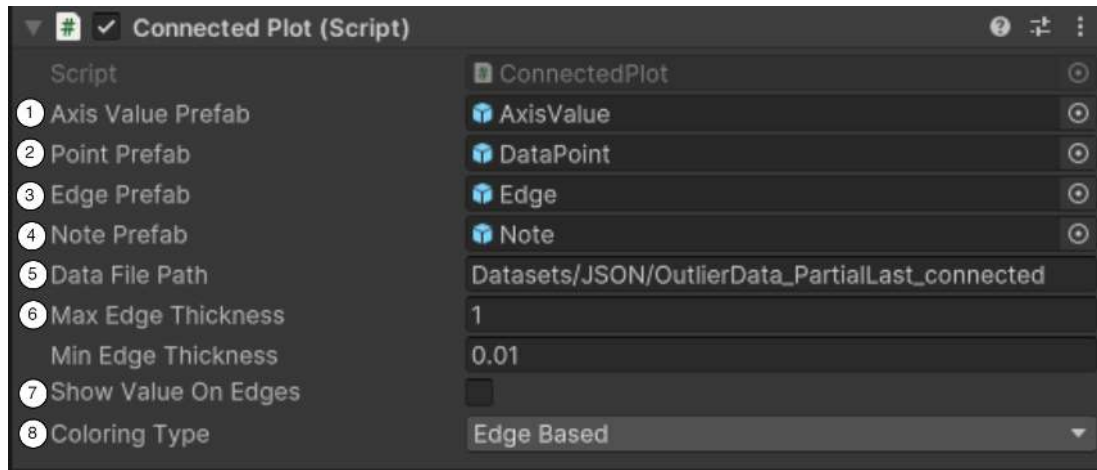


Figure 42: Connected plot in Unity Inspector.

1. **Axis Value Prefab:** The prefab that is used for the axis values.
2. **Data Point Prefab:** The prefab that represents individual data points in the visualization.
3. **Edge Prefab:** The prefab that represents the line that connects the data points.
4. **Note Prefab:** The prefab that represents the prefab used for standalone labels.
5. **Edge Thickness:** Sets the max and min thickness of the edge that connects nodes. The input file defines a value from 0-1, which is then mapped to these values.
6. **Show Value on Edges:** Determines if the thickness value is presented on the edge.
7. **Coloring Type:** Choose between Edge based, or Thickness Based to color the edges as described in Section ??.

1.3. Input file formats

1.3.1. Prefab Plot

Listing 1: Prefab Plot Sample Input File

```
1 {
2   /* Data Points. Each Data Point has to have the same size of points, but
3     they can be any size. */
4   "points": [
5     [
6       1.0,
7       2.0,
8       ...
9       n
10    ],
11    [
12      10.0,
13      20.0,
14      ...
15      n
16    ],
17    ...
18  ],
19  /* Labels for each data point. Size must be the same as points. */
20  "labels": [
21    "label1",
22    "label2",
23    ...
24  ],
25  /* Colors for each data point. Size must be the same as points. Value a is
26     transparency. */
27  "colors": [
28    [
29      /* r */ 0,
30      /* g */ 1,
31      /* b */ 0,
32      /* a */ 1
33    ],
34    ...
35  ]
36 }
```

1.3.2. Connected Plot

Listing 2: Connected Plot Sample Input File

```

1 {
2   /* Data Points. Each point must have a UNIQUE label. */
3   "nodes": {
4     "label1": [
5       1.0,
6       2.0,
7       3.0
8     ],
9     "label2": [
10      10.0,
11      20.0,
12      30.0
13    ],
14    ...
15  },
16  /* Edges of connecting Nodes. Each Label used must be defined above in nodes
17    .*/
18  "edges": [
19    ["label1", "label2"],
20    ...
21  ],
22  /* Edge Thickness of each edge. Must be the same size as edges. Must only
23    contain values from 0-1.
24  "edge_thickness": [
25    1.0,
26    0.5,
27    ...
28  ],
29  /* Colors for each data point. Size must be the same as nodes. Value a is
30    transparency. */
31  "colors": [
32    [
33      /* r */ 0,
34      /* g */ 1,
35      /* b */ 0,
36      /* a */ 1
37    ],
38    ...
39  ],
40  /* Standalone Labels. The name of the note is what will be displayed to the
41    user.
42  notes: {
43    "Note1": [
44      1.0,
45      2.0,
46      3.0
47    ],
48    ...
49  }
50 }

```

B Appendix: ImmerseXR Unity Package Class Reference

By implementing the following abstract classes and functions you can create your own dataset of datapoints, and thus your own visualization to suit your needs.

Axis Enum

Represents the axes in a 3D space, including a 'Root' for the base or origin.

ColumnInfo Class

Represents information about a data column, including its index, title, and description.

Utilities Class

A class containing utility methods for vector manipulation and object movement in a 3D space.

GetAxisFromVector Method

Extracts the specific axis value from a given `Vector3`. The axis to be extracted is specified by the `Axis` enum.

MoveOverSeconds Method

A coroutine method that moves a `GameObject` to a specified position over a given duration. The movement can be in local or global space, as indicated by the `isLocalPosition` parameter.

PlotManager<T> Class

An abstract class inheriting from `MonoBehaviour`, designed to manage plot data in Unity. The class is generic, with `T` being a type derived from `DataPoint`.

Public Fields

- **GameObject axisValuePrefab**: Prefab for axis values.
- **Vector3 positionDimensions**: Represents dimensions of the position in a 3D space. (Hidden in Inspector)
- **DataSet<T> dataSet**: The dataset being managed.
- **bool isValidData**: Flag to check if the data is valid. (Hidden in Inspector)
- **bool isAnimating**: Flag to check if there is an ongoing animation. (Hidden in Inspector)

Public Methods

Method Descriptions

FitDataToViewport Adjusts the plot's scale and position to fit within the viewport, ensuring all data is visible.

EnterNewElementMode Enters a special mode called 'Data Dive', which may offer different interaction or visualization options.

ExitNewElementMode Exits the 'Data Dive' mode, returning to the standard visualization state.

SetValidityOfData (protected) Sets a flag indicating whether the currently loaded data is valid and can be displayed.

Abstract Methods

These methods are declared in the 'PlotManager<T>' class but must be implemented in derived classes.

Method Descriptions

DisplayData (Abstract Method): Implement this method to define how data should be displayed after it is loaded. The method is responsible for rendering or visualizing the data on the plot.

DestroyData (Abstract Method): Implement this method to define how to clear or remove data from the plot. This is typically used for cleanup or preparation before loading new data.

UpdateData (Abstract Method): Implement this method to define how the plot data should be updated. This typically involves clearing the existing data (calling `DestroyData`) and then loading or displaying new data (calling `DisplayData`).

DataSet<T> Class

An abstract class designed for managing and processing datasets, where `T` is a type that inherits from `DataPoint`.

Properties

- **T[] data:** An array of data points of type `T`.
- **Dictionary<string, List<float>> notes:** A dictionary to store notes related to the dataset.

Constructor

The protected constructor, ensuring that the class cannot be instantiated directly but only through derived classes.

Abstract Methods

ParseDataSet Parses the provided CSV dataset into the internal data structure.

IsValidDataSet Checks whether the given dataset string is valid according to the specific implementation.

Public Methods

GetMax & GetMin Returns the maximum or minimum `DataPoint` in the dataset based on the specified axis and dimensions.

GetMaxValue & GetMinValue Retrieves the maximum or minimum value from the dataset for a specified axis and position dimensions.

ToString Provides a string representation of the dataset, listing all data points.

C Appendix: Biology Use Case Files & Scripts

3.1. CSV Input File

The full CSV Input file that includes experiments, n12, n14, n15, n22.

```
1 X,Y,Z,label,color 381 12,3,6,n15_3 Day#17,"[0, 0, 1, 1]" 701 5,0,12,n14_1 Day#3,"[0.5, 1, 0.5, 1]"
2 7,1,0,n12_1 Day#7,"[0, 1, 0, 1]" 382 13,7,0,n15_3 Day#13,"[0, 0, 1, 1]" 702 5,1,12,n14_1 Day#5,"[0.5, 1, 0.5, 1]"
3 13,1,0,n12_1 Day#13,"[0, 1, 0, 1]" 383 14,18,6,n15_3 Day#14,"[0, 0, 1, 1]" 703 10,2,12,n14_1 Day#10,"[0.5, 1, 0.5, 1]"
4 14,1,0,n12_1 Day#14,"[0, 1, 0, 1]" 384 15,2,0,n15_3 Day#15,"[0, 0, 1, 1]" 704 13,2,12,n14_1 Day#13,"[0.5, 1, 0.5, 1]"
5 10,4,0,n12_1 Day#10,"[0, 1, 0, 1]" 385 16,3,0,n15_3 Day#16,"[0, 0, 1, 1]" 705 14,4,12,n14_1 Day#14,"[0.5, 1, 0.5, 1]"
6 18,5,0,n12_1 Day#18,"[0, 1, 0, 1]" 386 17,7,0,n15_3 Day#17,"[0, 0, 1, 1]" 706 15,7,12,n14_1 Day#15,"[0.5, 1, 0.5, 1]"
7 21,2,0,n12_1 Day#21,"[0, 1, 0, 1]" 387 18,6,0,n15_3 Day#18,"[0, 0, 1, 1]" 707 10,3,12,n14_1 Day#10,"[0.5, 1, 0.5, 1]"
8 22,5,0,n12_1 Day#22,"[0, 1, 0, 1]" 388 19,5,6,n15_3 Day#19,"[0, 0, 1, 1]" 708 17,5,12,n14_1 Day#17,"[0.5, 1, 0.5, 1]"
9 23,20,0,n12_1 Day#23,"[0, 1, 0, 1]" 389 20,8,6,n15_3 Day#20,"[0, 0, 1, 1]" 709 18,0,12,n14_1 Day#18,"[0.5, 1, 0.5, 1]"
10 25,3,0,n12_1 Day#25,"[0, 1, 0, 1]" 390 21,2,0,n15_3 Day#21,"[0, 0, 1, 1]" 710 19,11,12,n14_1 Day#19,"[0.5, 1, 0.5, 1]"
11 26,2,0,n12_1 Day#26,"[0, 1, 0, 1]" 391 22,5,6,n15_3 Day#22,"[0, 0, 1, 1]" 711 20,8,12,n14_1 Day#20,"[0.5, 1, 0.5, 1]"
12 27,4,0,n12_1 Day#27,"[0, 1, 0, 1]" 392 23,3,6,n15_3 Day#23,"[0, 0, 1, 1]" 712 21,11,12,n14_1 Day#21,"[0.5, 1, 0.5, 1]"
13 28,2,0,n12_1 Day#28,"[0, 1, 0, 1]" 393 24,0,6,n15_3 Day#24,"[0, 0, 1, 1]" 713 22,10,12,n14_1 Day#22,"[0.5, 1, 0.5, 1]"
14 31,4,0,n12_1 Day#31,"[0, 1, 0, 1]" 394 26,1,6,n15_3 Day#26,"[0, 0, 1, 1]" 714 23,6,12,n14_1 Day#23,"[0.5, 1, 0.5, 1]"
15 5,1,1,n12_2 Day#5,"[0, 1, 0, 1]" 395 28,1,6,n15_3 Day#28,"[0, 0, 1, 1]" 715 24,5,12,n14_1 Day#24,"[0.5, 1, 0.5, 1]"
16 7,1,1,n12_2 Day#7,"[0, 1, 0, 1]" 396 10,1,7,n15_4 Day#10,"[0, 0, 1, 1]" 716 25,3,12,n14_1 Day#25,"[0.5, 1, 0.5, 1]"
17 12,1,1,n12_2 Day#12,"[0, 1, 0, 1]" 397 11,4,7,n15_4 Day#11,"[0, 0, 1, 1]" 717 26,4,12,n14_1 Day#26,"[0.5, 1, 0.5, 1]"
18 13,2,1,n12_2 Day#13,"[0, 1, 0, 1]" 398 12,3,7,n15_4 Day#12,"[0, 0, 1, 1]" 718 28,1,12,n14_1 Day#28,"[0.5, 1, 0.5, 1]"
19 14,5,1,n12_2 Day#14,"[0, 1, 0, 1]" 399 13,7,7,n15_4 Day#13,"[0, 0, 1, 1]" 719 3,1,13,n14_2 Day#3,"[0.5, 1, 0.5, 1]"
20 16,34,1,n12_2 Day#16,"[0, 1, 0, 1]" 400 14,16,7,n15_4 Day#14,"[0, 0, 1, 1]" 720 6,1,13,n14_2 Day#6,"[0.5, 1, 0.5, 1]"
21 18,0,1,n12_2 Day#18,"[0, 1, 0, 1]" 401 15,3,7,n15_4 Day#15,"[0, 0, 1, 1]" 721 7,1,13,n14_2 Day#7,"[0.5, 1, 0.5, 1]"
22 21,5,1,n12_2 Day#21,"[0, 1, 0, 1]" 402 16,7,7,n15_4 Day#16,"[0, 0, 1, 1]" 722 8,1,13,n14_2 Day#8,"[0.5, 1, 0.5, 1]"
23 22,1,1,n12_2 Day#22,"[0, 1, 0, 1]" 403 17,3,7,n15_4 Day#17,"[0, 0, 1, 1]" 723 10,1,13,n14_2 Day#10,"[0.5, 1, 0.5, 1]"
24 23,5,1,n12_2 Day#23,"[0, 1, 0, 1]" 404 18,4,7,n15_4 Day#18,"[0, 0, 1, 1]" 724 12,1,13,n14_2 Day#12,"[0.5, 1, 0.5, 1]"
25 25,1,1,n12_2 Day#25,"[0, 1, 0, 1]" 405 19,6,7,n15_4 Day#19,"[0, 0, 1, 1]" 725 15,3,13,n14_2 Day#15,"[0.5, 1, 0.5, 1]"
26 26,1,1,n12_2 Day#26,"[0, 1, 0, 1]" 406 20,7,7,n15_4 Day#20,"[0, 0, 1, 1]" 726 16,1,13,n14_2 Day#16,"[0.5, 1, 0.5, 1]"
27 27,6,1,n12_2 Day#27,"[0, 1, 0, 1]" 407 21,7,7,n15_4 Day#21,"[0, 0, 1, 1]" 727 17,6,13,n14_2 Day#17,"[0.5, 1, 0.5, 1]"
28 28,2,1,n12_2 Day#28,"[0, 1, 0, 1]" 408 22,6,7,n15_4 Day#22,"[0, 0, 1, 1]" 728 18,12,13,n14_2 Day#18,"[0.5, 1, 0.5, 1]"
29 29,4,1,n12_2 Day#29,"[0, 1, 0, 1]" 409 23,3,7,n15_4 Day#23,"[0, 0, 1, 1]" 729 19,10,13,n14_2 Day#19,"[0.5, 1, 0.5, 1]"
30 30,1,1,n12_2 Day#30,"[0, 1, 0, 1]" 410 25,11,7,n15_4 Day#25,"[0, 0, 1, 1]" 730 20,12,13,n14_2 Day#20,"[0.5, 1, 0.5, 1]"
31 31,1,1,n12_2 Day#31,"[0, 1, 0, 1]" 411 26,1,7,n15_4 Day#26,"[0, 0, 1, 1]" 731 21,5,13,n14_2 Day#21,"[0.5, 1, 0.5, 1]"
32 32,1,1,n12_2 Day#32,"[0, 1, 0, 1]" 412 27,1,7,n15_4 Day#27,"[0, 0, 1, 1]" 732 22,12,13,n14_2 Day#22,"[0.5, 1, 0.5, 1]"
33 2,1,2,n12_3 Day#2,"[0, 1, 0, 1]" 413 28,7,7,n15_4 Day#28,"[0, 0, 1, 1]" 733 23,4,13,n14_2 Day#23,"[0.5, 1, 0.5, 1]"
34 4,3,2,n12_3 Day#4,"[0, 1, 0, 1]" 414 29,1,7,n15_4 Day#29,"[0, 0, 1, 1]" 734 24,4,13,n14_2 Day#24,"[0.5, 1, 0.5, 1]"
35 5,2,2,n12_3 Day#5,"[0, 1, 0, 1]" 415 30,1,7,n15_4 Day#30,"[0, 0, 1, 1]" 735 25,1,13,n14_2 Day#25,"[0.5, 1, 0.5, 1]"
36 6,1,2,n12_3 Day#6,"[0, 1, 0, 1]" 416 8,1,8,n22_1 Day#8,"[0, 1, 1, 1]" 736 26,5,13,n14_2 Day#26,"[0.5, 1, 0.5, 1]"
37 7,3,2,n12_3 Day#7,"[0, 1, 0, 1]" 417 12,1,8,n22_1 Day#12,"[0, 1, 1, 1]" 737 2,3,14,n14_3 Day#2,"[0.5, 1, 0.5, 1]"
38 12,1,2,n12_3 Day#12,"[0, 1, 0, 1]" 418 13,1,8,n22_1 Day#13,"[0, 1, 1, 1]" 738 6,1,14,n14_3 Day#6,"[0.5, 1, 0.5, 1]"
39 13,2,2,n12_3 Day#13,"[0, 1, 0, 1]" 419 14,4,8,n22_1 Day#14,"[0, 1, 1, 1]" 739 7,1,14,n14_3 Day#7,"[0.5, 1, 0.5, 1]"
40 16,5,2,n12_3 Day#16,"[0, 1, 0, 1]" 420 16,5,8,n22_1 Day#16,"[0, 1, 1, 1]" 740 10,1,14,n14_3 Day#10,"[0.5, 1, 0.5, 1]"
41 18,0,2,n12_3 Day#18,"[0, 1, 0, 1]" 421 17,12,8,n22_1 Day#17,"[0, 1, 1, 1]" 741 11,1,14,n14_3 Day#11,"[0.5, 1, 0.5, 1]"
42 21,5,2,n12_3 Day#21,"[0, 1, 0, 1]" 422 18,10,8,n22_1 Day#18,"[0, 1, 1, 1]" 742 12,0,14,n14_3 Day#12,"[0.5, 1, 0.5, 1]"
43 22,2,2,n12_3 Day#22,"[0, 1, 0, 1]" 423 19,11,8,n22_1 Day#19,"[0, 1, 1, 1]" 743 13,3,14,n14_3 Day#13,"[0.5, 1, 0.5, 1]"
44 23,5,2,n12_3 Day#23,"[0, 1, 0, 1]" 424 20,19,8,n22_1 Day#20,"[0, 1, 1, 1]" 744 14,3,14,n14_3 Day#14,"[0.5, 1, 0.5, 1]"
45 25,2,2,n12_3 Day#25,"[0, 1, 0, 1]" 425 21,4,8,n22_1 Day#21,"[0, 1, 1, 1]" 745 15,4,14,n14_3 Day#15,"[0.5, 1, 0.5, 1]"
46 26,2,2,n12_3 Day#26,"[0, 1, 0, 1]" 426 22,5,8,n22_1 Day#22,"[0, 1, 1, 1]" 746 16,7,14,n14_3 Day#16,"[0.5, 1, 0.5, 1]"
47 28,3,2,n12_3 Day#28,"[0, 1, 0, 1]" 427 23,8,8,n22_1 Day#23,"[0, 1, 1, 1]" 747 17,6,14,n14_3 Day#17,"[0.5, 1, 0.5, 1]"
48 30,3,2,n12_3 Day#30,"[0, 1, 0, 1]" 428 24,4,8,n22_1 Day#24,"[0, 1, 1, 1]" 748 18,8,14,n14_3 Day#18,"[0.5, 1, 0.5, 1]"
49 4,1,3,n12_4 Day#4,"[0, 1, 0, 1]" 429 25,7,8,n22_1 Day#25,"[0, 1, 1, 1]" 749 19,15,14,n14_3 Day#19,"[0.5, 1, 0.5, 1]"
50 6,1,3,n12_4 Day#6,"[0, 1, 0, 1]" 430 26,3,8,n22_1 Day#26,"[0, 1, 1, 1]" 750 20,8,14,n14_3 Day#20,"[0.5, 1, 0.5, 1]"
51 12,1,3,n12_4 Day#12,"[0, 1, 0, 1]" 431 27,2,8,n22_1 Day#27,"[0, 1, 1, 1]" 751 21,7,14,n14_3 Day#21,"[0.5, 1, 0.5, 1]"
52 14,3,3,n12_4 Day#14,"[0, 1, 0, 1]" 432 8,3,9,n22_2 Day#8,"[0, 1, 1, 1]" 752 22,7,14,n14_3 Day#22,"[0.5, 1, 0.5, 1]"
53 16,11,3,n12_4 Day#16,"[0, 1, 0, 1]" 433 9,2,9,n22_2 Day#9,"[0, 1, 1, 1]" 753 23,3,14,n14_3 Day#23,"[0.5, 1, 0.5, 1]"
54 18,2,3,n12_4 Day#18,"[0, 1, 0, 1]" 434 11,2,9,n22_2 Day#11,"[0, 1, 1, 1]" 754 24,4,14,n14_3 Day#24,"[0.5, 1, 0.5, 1]"
55 21,12,3,n12_4 Day#21,"[0, 1, 0, 1]" 435 13,1,9,n22_2 Day#13,"[0, 1, 1, 1]" 755 25,2,14,n14_3 Day#25,"[0.5, 1, 0.5, 1]"
56 22,7,3,n12_4 Day#22,"[0, 1, 0, 1]" 436 14,2,9,n22_2 Day#14,"[0, 1, 1, 1]" 756 26,1,14,n14_3 Day#26,"[0.5, 1, 0.5, 1]"
57 23,7,3,n12_4 Day#23,"[0, 1, 0, 1]" 437 15,2,9,n22_2 Day#15,"[0, 1, 1, 1]" 757 5,1,15,n14_4 Day#5,"[0.5, 1, 0.5, 1]"
58 24,3,3,n12_4 Day#24,"[0, 1, 0, 1]" 438 16,4,9,n22_2 Day#16,"[0, 1, 1, 1]" 758 6,2,15,n14_4 Day#6,"[0.5, 1, 0.5, 1]"
59 26,2,3,n12_4 Day#26,"[0, 1, 0, 1]" 439 17,10,9,n22_2 Day#17,"[0, 1, 1, 1]" 759 10,1,15,n14_4 Day#10,"[0.5, 1, 0.5, 1]"
60 28,2,3,n12_4 Day#28,"[0, 1, 0, 1]" 440 18,4,9,n22_2 Day#18,"[0, 1, 1, 1]" 760 13,1,15,n14_4 Day#13,"[0.5, 1, 0.5, 1]"
61 29,1,3,n12_4 Day#29,"[0, 1, 0, 1]" 441 19,14,9,n22_2 Day#19,"[0, 1, 1, 1]" 761 14,2,15,n14_4 Day#14,"[0.5, 1, 0.5, 1]"
62 11,0,4,n15_1 Day#11,"[0, 0, 1, 1]" 442 20,5,9,n22_2 Day#20,"[0, 1, 1, 1]" 762 15,4,15,n14_4 Day#15,"[0.5, 1, 0.5, 1]"
63 12,1,4,n15_1 Day#12,"[0, 0, 1, 1]" 443 21,8,9,n22_2 Day#21,"[0, 1, 1, 1]" 763 16,2,15,n14_4 Day#16,"[0.5, 1, 0.5, 1]"
64 13,2,4,n15_1 Day#13,"[0, 0, 1, 1]" 444 22,6,9,n22_2 Day#22,"[0, 1, 1, 1]" 764 17,6,15,n14_4 Day#17,"[0.5, 1, 0.5, 1]"
65 14,2,4,n15_1 Day#14,"[0, 0, 1, 1]" 445 23,4,9,n22_2 Day#23,"[0, 1, 1, 1]" 765 18,4,15,n14_4 Day#18,"[0.5, 1, 0.5, 1]"
66 15,4,4,n15_1 Day#15,"[0, 0, 1, 1]" 446 24,0,9,n22_2 Day#24,"[0, 1, 1, 1]" 766 19,6,15,n14_4 Day#19,"[0.5, 1, 0.5, 1]"
67 16,2,4,n15_1 Day#16,"[0, 0, 1, 1]" 447 25,2,9,n22_2 Day#25,"[0, 1, 1, 1]" 767 20,9,15,n14_4 Day#20,"[0.5, 1, 0.5, 1]"
68 17,2,4,n15_1 Day#17,"[0, 0, 1, 1]" 448 26,1,9,n22_2 Day#26,"[0, 1, 1, 1]" 768 21,6,15,n14_4 Day#21,"[0.5, 1, 0.5, 1]"
69 18,4,4,n15_1 Day#18,"[0, 0, 1, 1]" 449 27,1,9,n22_2 Day#27,"[0, 1, 1, 1]" 769 22,12,15,n14_4 Day#22,"[0.5, 1, 0.5, 1]"
70 19,10,4,n15_1 Day#19,"[0, 0, 1, 1]" 450 9,3,10,n22_3 Day#9,"[0, 1, 1, 1]" 770 23,9,15,n14_4 Day#23,"[0.5, 1, 0.5, 1]"
71 20,8,4,n15_1 Day#20,"[0, 0, 1, 1]" 451 12,3,10,n22_3 Day#12,"[0, 1, 1, 1]" 771 24,4,15,n14_4 Day#24,"[0.5, 1, 0.5, 1]"
72 21,15,4,n15_1 Day#21,"[0, 0, 1, 1]" 452 13,3,10,n22_3 Day#13,"[0, 1, 1, 1]" 772 25,3,15,n14_4 Day#25,"[0.5, 1, 0.5, 1]"
73 22,10,4,n15_1 Day#22,"[0, 0, 1, 1]" 453 14,2,10,n22_3 Day#14,"[0, 1, 1, 1]" 773 26,3,15,n14_4 Day#26,"[0.5, 1, 0.5, 1]"
74 23,4,4,n15_1 Day#23,"[0, 0, 1, 1]" 454 15,1,10,n22_3 Day#15,"[0, 1, 1, 1]" 774 27,1,15,n14_4 Day#27,"[0.5, 1, 0.5, 1]"
75 24,0,4,n15_1 Day#24,"[0, 0, 1, 1]" 455 16,6,10,n22_3 Day#16,"[0, 1, 1, 1]"
76 25,0,4,n15_1 Day#25,"[0, 0, 1, 1]" 456 17,17,10,n22_3 Day#17,"[0, 1, 1, 1]"
77 27,3,4,n15_1 Day#27,"[0, 0, 1, 1]" 457 18,19,10,n22_3 Day#18,"[0, 1, 1, 1]"
78 28,3,4,n15_1 Day#28,"[0, 0, 1, 1]" 458 19,21,10,n22_3 Day#19,"[0, 1, 1, 1]"
79 29,1,4,n15_1 Day#29,"[0, 0, 1, 1]" 459 20,8,10,n22_3 Day#20,"[0, 1, 1, 1]"
80 10,3,5,n15_2 Day#10,"[0, 0, 1, 1]" 460 21,7,10,n22_3 Day#21,"[0, 1, 1, 1]"
81 11,1,5,n15_2 Day#11,"[0, 0, 1, 1]" 461 22,5,10,n22_3 Day#22,"[0, 1, 1, 1]"
82 12,2,5,n15_2 Day#12,"[0, 0, 1, 1]" 462 23,0,10,n22_3 Day#23,"[0, 1, 1, 1]"
83 13,3,5,n15_2 Day#13,"[0, 0, 1, 1]" 463 24,5,10,n22_3 Day#24,"[0, 1, 1, 1]"
84 14,10,5,n15_2 Day#14,"[0, 0, 1, 1]" 464 9,4,11,n22_4 Day#9,"[0, 1, 1, 1]"
85 15,3,5,n15_2 Day#15,"[0, 0, 1, 1]" 465 11,5,11,n22_4 Day#11,"[0, 1, 1, 1]"
86 16,5,5,n15_2 Day#16,"[0, 0, 1, 1]" 466 12,1,11,n22_4 Day#12,"[0, 1, 1, 1]"
87 17,2,5,n15_2 Day#17,"[0, 0, 1, 1]" 467 13,1,11,n22_4 Day#13,"[0, 1, 1, 1]"
88 18,3,5,n15_2 Day#18,"[0, 0, 1, 1]" 468 14,2,11,n22_4 Day#14,"[0, 1, 1, 1]"
89 19,4,5,n15_2 Day#19,"[0, 0, 1, 1]" 469 15,2,11,n22_4 Day#15,"[0, 1, 1, 1]"
90 20,6,5,n15_2 Day#20,"[0, 0, 1, 1]" 470 16,1,11,n22_4 Day#16,"[0, 1, 1, 1]"
91 21,10,5,n15_2 Day#21,"[0, 0, 1, 1]" 471 17,23,11,n22_4 Day#17,"[0, 1, 1, 1]"
92 22,6,5,n15_2 Day#22,"[0, 0, 1, 1]" 472 18,8,11,n22_4 Day#18,"[0, 1, 1, 1]"
93 23,7,5,n15_2 Day#23,"[0, 0, 1, 1]" 473 19,13,11,n22_4 Day#19,"[0, 1, 1, 1]"
94 24,4,5,n15_2 Day#24,"[0, 0, 1, 1]" 474 20,17,11,n22_4 Day#20,"[0, 1, 1, 1]"
95 25,7,5,n15_2 Day#25,"[0, 0, 1, 1]" 475 21,13,11,n22_4 Day#21,"[0, 1, 1, 1]"
96 28,1,5,n15_2 Day#28,"[0, 0, 1, 1]" 476 22,8,11,n22_4 Day#22,"[0, 1, 1, 1]"
97 29,1,5,n15_2 Day#29,"[0, 0, 1, 1]" 477 23,4,11,n22_4 Day#23,"[0, 1, 1, 1]"
98 9,4,6,n15_3 Day#9,"[0, 0, 1, 1]" 478 24,4,11,n22_4 Day#24,"[0, 1, 1, 1]"
99 10,4,6,n15_3 Day#10,"[0, 0, 1, 1]" 479 25,5,11,n22_4 Day#25,"[0, 1, 1, 1]"
100 11,3,6,n15_3 Day#11,"[0, 0, 1, 1]" 480 26,8,11,n22_4 Day#26,"[0, 1, 1, 1]"
```

Figure 43: Contents of the input CSV File.

3.2. Python Scripts

Listing 3: timelines_csv_to_json.py

```
import csv
import json
import ast
import argparse
import timelines_csv_to_json_scoringFunctions as scoringFunctions
from sklearn.preprocessing import minmax_scale
import numpy as np

def convert_csv_to_connected_json(csv_file , json_file , json_non_connected_file):

    data = { 'points': [], 'labels': [], 'colors': [] }
    all_timeseries = []

    with open(csv_file , 'r') as file:
        csv_reader = csv.reader(file)
        next(csv_reader) # Skip the header row

        for row in csv_reader:
            x, y, z, label , color = row
            z = float(z) * z_multiplier
            data[ 'points' ].append([ float(x), float(y), z ])
            data[ 'labels' ].append(label)
            data[ 'colors' ].append(ast.literal_eval(color))

    # Use json dump to write to file and re-read it to ensure the data is correct
    with open(json_non_connected , 'w') as file:
        json.dump(data , file)

    with open(json_non_connected , 'r') as f:
        data = json.load(f)

    # Create the nodes, edges and edge thicknesses
    nodes = {}
    edges = []
    edge_thickness = []
```

```
# Colors for different uses, one for displaying the plot with box
colors_quantiles = data['colors']
colors_zscores = [[1, 1, 1, 1]] * len(colors_quantiles)

# Create the labels included in the input file
labels = data['labels']

# Create a list of all the labels
dataset_labels = []

# Include all points from the data set
points = data['points']

# For splitting the time series and processing each one separately
current_timeseries = []
current_substring = ""

for i in range(len(labels)):
    label = labels[i]
    point = points[i]

    # Split Experiments and Groups by looking at the label
    # Could be improved by looking at Z, but used this algorithm to
    # the coloring too at some point so it stayed like this
    current_substring = label.split('_')[0]

    if i > 1:
        prev_label = labels[i-1]
        prev_substring = prev_label.split('_')[0]

        if current_substring != prev_substring:
            all_timeseries.append(current_timeseries.copy())
            current_timeseries = []
            dataset_labels.append(prev_substring)

    current_timeseries.append(point)

# Connect the nodes to simulate time series
```

```

nodes[label] = point

if i < len(labels) - 1:
    next_label = labels[i+1]
    next_substring = next_label.split('_')[0]
    if current_substring == next_substring:
        edge = [label, next_label]
        edges.append(edge)
        edge_thickness.append(0.2)

dataset_labels.append(current_substring)
all_timeseries.append(current_timeseries.copy())

# Create a temporary list to hold the scores for the partial timeline
partial_scores = []

for i, timeseries in enumerate(all_timeseries):
    # Calculate the score up to each point in the time series
    for j in range(len(timeseries)):
        # Ensure that there are at least 3 points to perform KMean
        if j+1 >= 3:
            partial_timeline = [sublist[:j] for sublist in all_timeseries]

            # Scoring results from weighted result of knn and novel
            score = scoringFunctions.score_sum(i, partial_timeline)

            partial_scores.append(score)
        else:
            # If we have less than 3 points, append a default score
            partial_scores.append(0) # Assuming 0 is the default

partial_scores = normalize(partial_scores)

k = 0
# Now, color the nodes based on the partial scores
for score in partial_scores:
    colors_zscores[k] = interpolate_color([1, 1, 0, 1], [1, 0, 0, 1])
    k += 1

```

```

nodes_quantiles = nodes.copy()
nodes_zscores = nodes.copy()

edges_quantiles = edges.copy()
edges_zscores = edges.copy()

edge_thickness_quantiles = edge_thickness.copy()
edge_thickness_zscores = edge_thickness.copy()

colors_quantiles = colors_zscores.copy()

# Import quantile data and generate box plots
with open(additional_csv_file, 'r') as file:
    csv_reader = csv.reader(file)
    next(csv_reader) # Skip the header row

    for row in csv_reader:
        x, min_val, first_val, median_val, fourth_val, max_val, iq

        y_values = [float(min_val), float(first_val), float(median_val), float(fourth_val), float(max_val)]
        y_labels = ["Min", "1st", "Median", "4th", "Max"]

        for y, y_label in zip(y_values, y_labels):
            point_label = f"{dataset}_{y_label}"
            point = [float(x), y, float(z) * z_multiplier]
            nodes_quantiles[point_label] = point
            if y_label in ["1st", "4th"]:
                original_color = ast.literal_eval(color)
                lighter_color = [(c + 0.2) if c <= 0.8 else 1.0 for c in original_color]
                colors_quantiles.append(lighter_color)
            else:
                colors_quantiles.append(ast.literal_eval(color))

        edges_quantiles.append([f"{dataset}_Min", f"{dataset}_1st", f"{dataset}_4th", f"{dataset}_Max"])
        edges_quantiles.append([f"{dataset}_1st", f"{dataset}_4th", f"{dataset}_Max", f"{dataset}_Min"])
        edges_quantiles.append([f"{dataset}_4th", f"{dataset}_Max", f"{dataset}_Min", f"{dataset}_1st"])
        edge_thickness_quantiles.extend([0.1, 1, 0.1])

# Generate Time series with Box plot Visualization

```

```

output_data = {'nodes': nodes_quantiles, 'edges': edges_quantiles,
with open(json_file, 'w') as f:
    json.dump(output_data, f)

# Generate Time series with Score Coloring Visualization
output_data_white = {'nodes': nodes_zscores, 'edges': edges_zscores}
with open(json_white, 'w') as f:
    json.dump(output_data_white, f)

def normalize(lst):
    min_value = min(lst)
    max_value = max(lst)
    range_value = max_value - min_value
    return [(value - min_value) / range_value for value in lst]

def interpolate_color(start_color, end_color, factor):
    # Ensure factor is within [0, 1]
    factor = max(0.0, min(1.0, factor))

    # Interpolate each RGB component separately
    interpolated_color = [start + (end - start) * factor for start, end in start_color, end_color]

    return interpolated_color

def main():
    parser = argparse.ArgumentParser(description='Convert_CSV_to_connected_JSON')
    parser.add_argument('csv_file', help='Input_CSV_file_name')
    parser.add_argument('json_file', help='Output_connected_JSON_file_name')
    parser.add_argument('json_non_connected', help='Output_non-connected_JSON_file_name')
    parser.add_argument('json_white', help='Output_connected_but_white_JSON_file_name')
    parser.add_argument('z_multiplier', type=float, help='Multiplier_for_z_scores')
    parser.add_argument('additional_csv_file', help='Additional_CSV_file_name')

    args = parser.parse_args()

    convert_csv_to_connected_json(args.csv_file, args.json_file, args.json_non_connected,
                                  args.json_white, args.z_multiplier, args.additional_csv_file)

if __name__ == '__main__':
    main()

```

Also used by `timelines_csv_to_json.py` to calculate the scores using various methods:

Listing 4: `timelines_csv_to_json_scoringFunctions.py`

```
import numpy as np
from sklearn.cluster import KMeans
from sklearn.metrics import pairwise_distances_argmin_min
import numpy as np

# Clustering based scoring
def score_clustering_based(target_index, all_series):
    # Ensure all series have the same length
    all_series = pad_series(all_series)

    # Extract only Y-values for KMeans
    all_values_y = [series[:,1] for series in all_series]

    # Check for consistent series lengths
    series_lengths = {len(series) for series in all_values_y}
    if len(series_lengths) > 1:
        raise ValueError("All_series_must_be_of_same_length_for_KMeans")

    kmeans = KMeans(n_clusters=3).fit(all_values_y)

    # Ensuring target_series Y-values are 2D array for pairwise_distances_argmin_min
    target_series_y = all_values_y[target_index].reshape(1, -1)

    closest, _ = pairwise_distances_argmin_min(kmeans.cluster_centers_, target_series_y)

    target_cluster = kmeans.labels_[closest[0]]

    within_cluster_distances = [calculate_distance(all_series[target_index], all_series[cluster])
                                for cluster in kmeans.labels_]

    outlier_score = np.mean(within_cluster_distances)

    return outlier_score / 100

def pad_series(all_series):
    max_length = max(len(series) for series in all_series)
```

```
padded_series = []
for series in all_series:
    series_length = len(series)
    # Ensure each series is a NumPy array
    series_np = np.array(series)
    # Generate the padding
    padding = np.zeros((max_length - series_length, series_np.shape[0]))
    # Add the padding to the original series
    padded = np.vstack([series_np, padding])
    padded_series.append(padded)
return padded_series

def calculate_distance(series1, series2):
    return np.linalg.norm(series1 - series2)

# Outlier based scoring
def score_outlier_based(target_index, all_series):
    # Create stats
    stats_series = []
    for series in all_series:
        scores = time_series_statistics(series)
        stats_series.append(scores)
    stats_series = np.array(stats_series)

    # Get low and upper quartiles of stats
    low_quartiles, upper_quartiles = evaluate_quartiles(stats_series)

    outlier_rank = check_for_outliers(stats_series, low_quartiles, upper_quartiles)

    return outlier_rank[target_index]

def check_for_outliers(data, low_quartile, upper_quartile, alpha=0.2):
    how_many_cols = len(data[0])

    final_col = [0] * len(data)

    for i in range(how_many_cols):
        curr_col = data[:, i]
        height_of_col = len(curr_col)
```

```
        for j in range(height_of_col):
            if (isOutlier(curr_col[j], upper_quartile[i], low_quartile[i],
                           final_col[j] += 1

    return final_col

def isOutlier(value, upper, low, alpha):
    iqr = upper - low
    lower_bound = low - alpha * iqr
    upper_bound = upper + alpha * iqr

    return (value < lower_bound) | (value > upper_bound)

def evaluate_quartiles(data_set):
    num_values_per_sample = len(data_set[0])
    low_quartiles = []
    upper_quartiles = []
    for i in range(num_values_per_sample):
        low_quartile, upper_quartile = np.percentile(data_set[:, i], [25, 75])
        low_quartiles.append(low_quartile)
        upper_quartiles.append(upper_quartile)
    return low_quartiles, upper_quartiles

def time_series_statistics(time_series):
    # Convert the input to a numpy array to ensure compatibility with
    time_series = np.array(time_series)

    # Calculate required statistics
    min_value = np.min(time_series)
    q1 = np.percentile(time_series, 25)
    median = np.median(time_series)
    q3 = np.percentile(time_series, 75)
    max_value = np.max(time_series)
    iqr = q3 - q1

    return [min_value, q1, median, q3, max_value, iqr]

# Sum of Clustering and outlier based scoring
def score_sum(target_series, all_series, w1, w2):
```

```
assert w1 + w2 <= 1
```

```
KMeansScore = score_clustering_based(target_series , all_series)
```

```
OutlierScore = score_outlier_based(target_series , all_series)
```

```
return w1 * KMeansScore + w2 * OutlierScore
```

Listing 5: timelines_csv_calculate_quantiles.py

```
import pandas as pd
```

```
import argparse
```

```
import numpy as np
```

```
def calculate_quantiles(input_file , output_file):
```

```
    # Load your data from the input CSV file
```

```
    data = pd.read_csv(input_file)
```

```
    # Create an empty DataFrame for the quantiles
```

```
    quantiles = pd.DataFrame(columns=['X', 'Min', '1st', 'Median', '4th'])
```

```
    # Get unique IDs from the data
```

```
    unique_ids = data['Z'].unique()
```

```
    # Loop through each ID and calculate quantiles and other statistics
```

```
    for id in unique_ids:
```

```
        id_data = data[data['Z'] == id]
```

```
        dataset = id_data['label'].iloc[0] # Assuming 'dataset' is a
```

```
        color = id_data['color'].iloc[0] # Assuming 'color' is a
```

```
        quantiles.loc[id] = [-1, # Constant value for X
```

```
                               id_data['Y'].min(),
```

```
    # Min
```

```
                               id_data['Y'].quantile(0.25),
```

```
    # 1st Quantile (25th percentile)
```

```
                               id_data['Y'].median(),
```

```
    # Median (50th percentile)
```

```
                               id_data['Y'].quantile(0.75),
```

```
    # 4th Quantile (75th percentile)
```

```
                               id_data['Y'].quantile(0.75) - id_data['Y']
```

```
    # IQR
```

```
                               np.exp(np.mean(np.log(id_data['Y']))),
```

```
    # Geometric Mean
```

```
                               id_data['Y'].max(),
```

```
    # Max
```

```
                                id ,
# Z                                dataset.split("_")[0] ,
# dataset                                color]
# color
    # Save the results to the output CSV file
    quantiles.to_csv(output_file , index=False)
if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Calculate_quantiles_')
    parser.add_argument('input_file' , help='Input_CSV_file')
    parser.add_argument('output_file' , help='Output_CSV_file')
    args = parser.parse_args()
    calculate_quantiles(args.input_file , args.output_file)
```